PUC Minas



Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Sumário

- Construção de serviços Web
  - Principais frameworks
  - Organização das rotas

Sumário

- Construção de serviços Web
  - **Principais frameworks**
  - Organização das rotas

Construção de serviços Web

- Principais frameworks
  - Na aula anterior (*AULA 01_Arquitetura de microsserviços*), construímos nossa primeira **API REST** utilizando o **Spring Framework** e a extensão **Spring Boot** para auxiliar na configuração inicial do projeto.
    - https://spring.io/guides/gs/rest-service/







Construção de serviços Web

- Principais frameworks
  - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
    - https://www.django-rest-framework.org/





Construção de serviços Web

- Principais frameworks
  - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
    - https://flask-restful.readthedocs.io/en/latest/



Flask RESTful



python™

Construção de serviços Web

- Principais frameworks
  - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
    - https://expressjs.com/en/5x/api.html
    - https://express-rest-api-generato.readthedocs.io/en/latest/

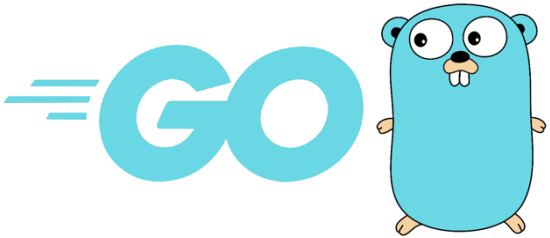






Construção de serviços Web

- Principais frameworks
  - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
    - https://go.dev/doc/tutorial/web-service-gin





Construção de serviços Web

- Principais frameworks
  - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
    - https://dotnet.microsoft.com/pt-br/apps/aspnet/apis





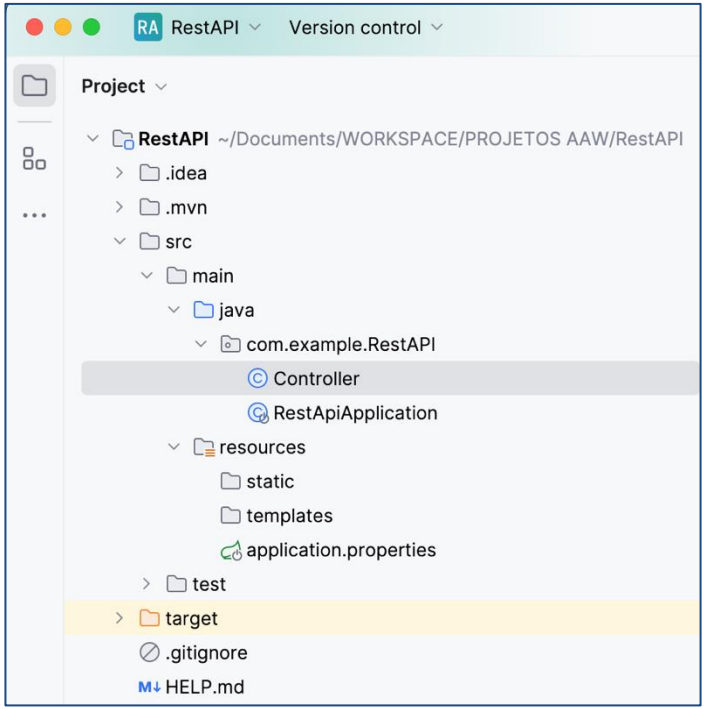
Sumário

- Construção de serviços Web
  - Principais frameworks
  - **Organização das rotas**

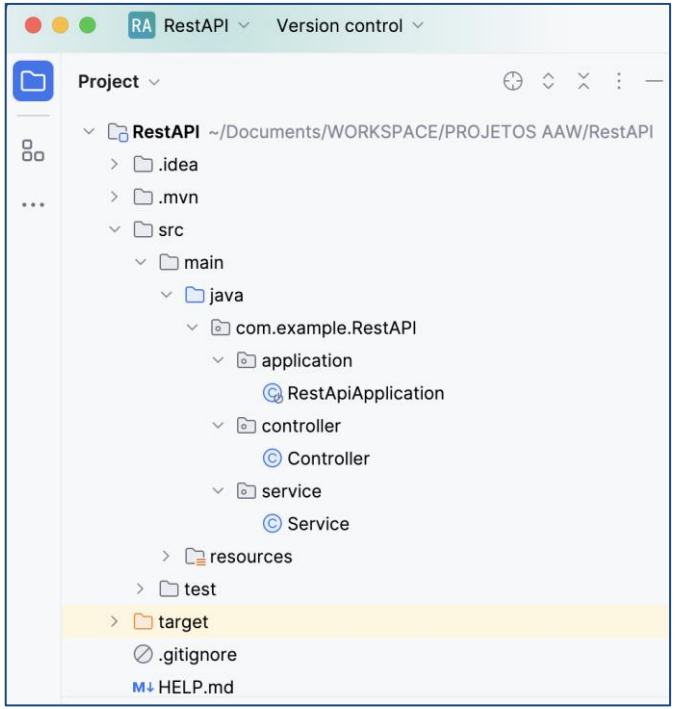
Organização das rotas

- Vamos agora aprimorar a arquitetura do nosso primeiro projeto de API REST

  e organizar melhor nossos **pacotes**, **serviços** e **rotas**.







Construção de serviços Web

- Organização das rotas
  - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
    - I) application
    - II) controller
    - III) service

    - Essa organização em camadas ou pacotes tem o objetivo de melhorar a modularidade, a manutenção e a compreensão do código.

Construção de serviços Web

- Organização das rotas
  - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
    - **I) application**
      - O pacote application geralmente contém a classe principal (Main) que inicia a aplicação.
      - Também pode incluir configurações de inicialização, configurações gerais do aplicativo e outros componentes relacionados à inicialização e configuração global.

Construção de serviços Web

- Organização das rotas
  - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
    - **II) controller**
      - O pacote controller é frequentemente usado para armazenar classes que atuam como controladores na arquitetura de uma aplicação.
      - Os controladores são responsáveis por receber as solicitações HTTP, interagir com os serviços subjacentes e retornar as respostas adequadas.

Construção de serviços Web

- Organização das rotas
  - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
    - **III) service**
      - O pacote service é destinado a conter as classes que implementam a lógica de negócio ou serviços.
      - Essas classes são responsáveis por manipular regras de negócio, realizar operações de processamento e interagir com o banco de dados, se necessário.

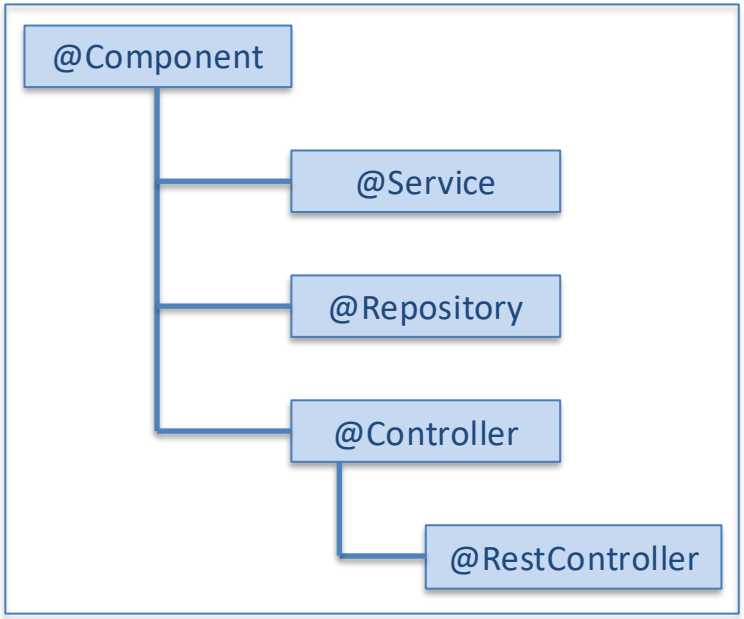
Construção de serviços Web

- Organização das rotas
  - Essa organização facilita a **manutenção**, a **escalabilidade** e a compreensão do código, pois cria uma separação clara de responsabilidades entre diferentes partes da aplicação. Ao seguir essa prática, a estrutura do projeto torna-se mais **modular** e **flexível**, facilitando a adição, remoção ou modificação de funcionalidades sem afetar outras partes da aplicação.

  - Essa abordagem também contribui para a **legibilidade do código**, pois permite que os desenvolvedores localizem rapidamente a funcionalidade relevante em seus respectivos pacotes.

Organização das rotas

- Futuramente, quando inserirmos um banco de dados na aplicação, teremos um quarto pacote (package), chamado **persistence**, **repository** ou **DAO** (Data Access Object).



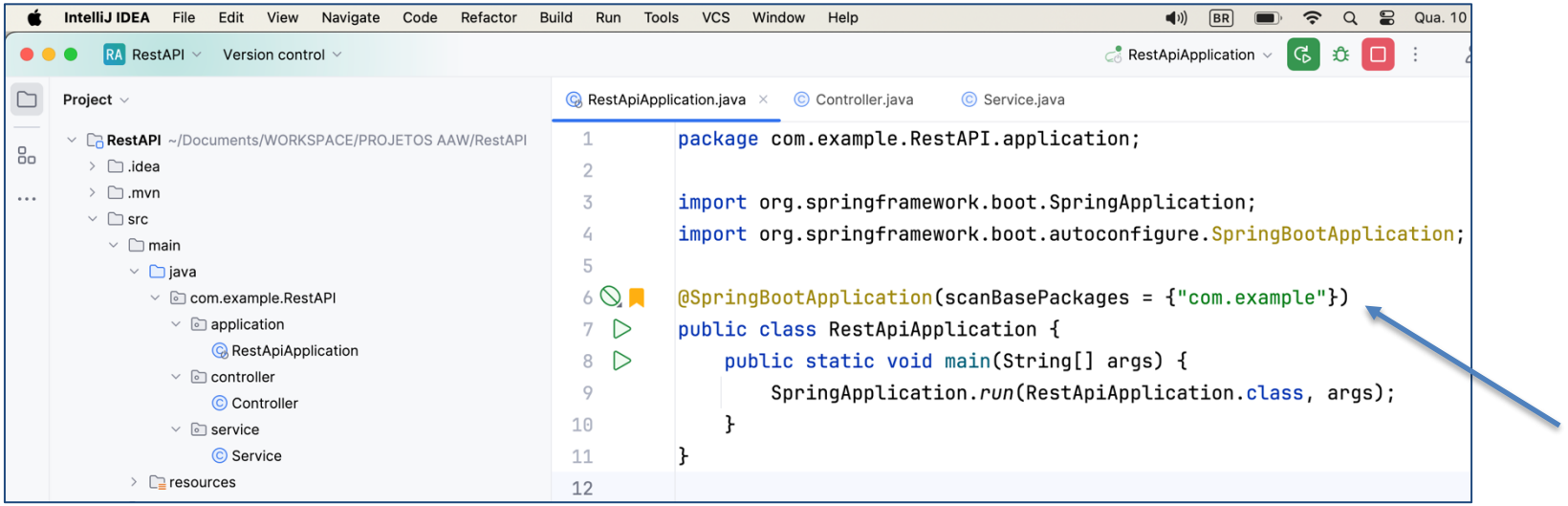
Construção de serviços Web

- Organização das rotas
  - A organização dos pacotes e a estrutura das rotas devem ser coerentes para proporcionar uma experiência de desenvolvimento mais consistente e facilitar a manutenção do código.

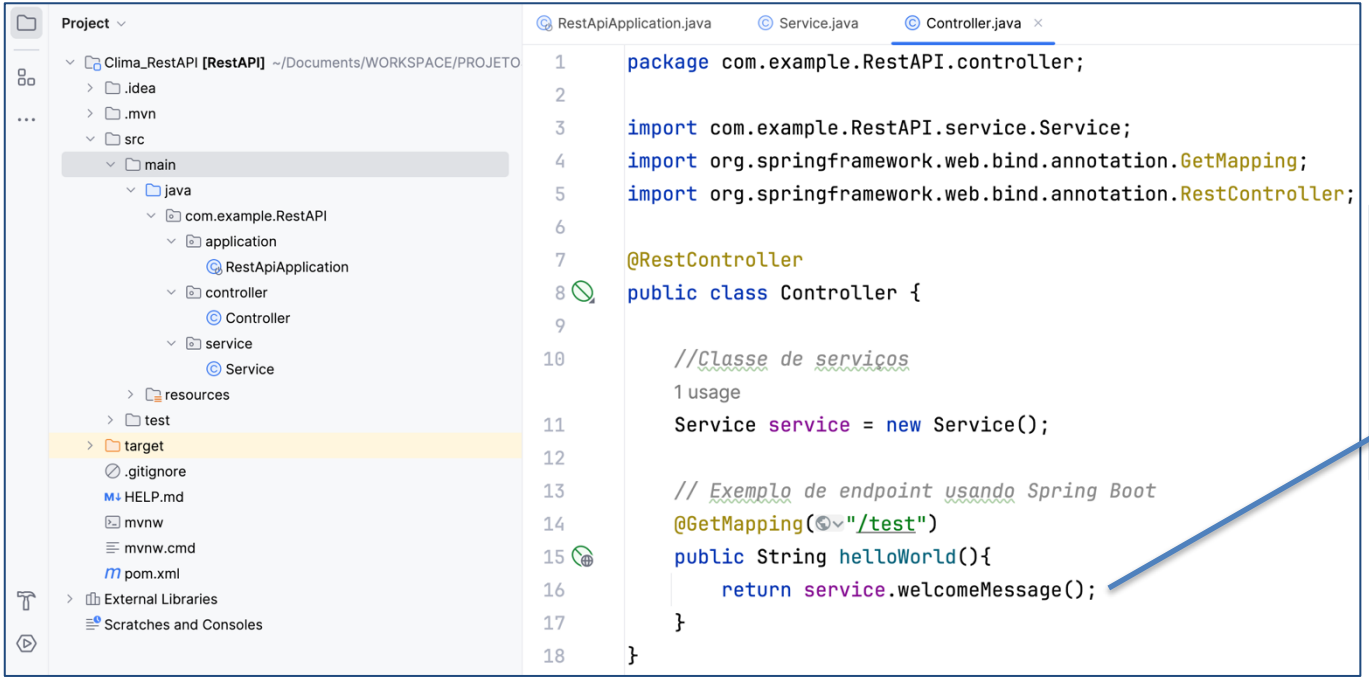
  - Quando um desenvolvedor trabalha em uma determinada funcionalidade, ele deve ser capaz de localizar facilmente os controladores associados e as rotas relacionadas nos pacotes apropriados. Isso melhora a legibilidade do código e a manutenção da aplicação.

Construção de serviços Web

Inclua o elemento: (scanBasePackages = {"com.example"}) à frente da annotation: @SpringBootApplication para que os nossos novos pacotes sejam encontrados.



Altere a classe **Controller** para exportar a lógica de negócio para uma classe **Service**. Assim o controlador fica responsável apenas por criar as rotas (/test), enquanto as regras de negócio ficarão em uma classe de serviços.





Construção de serviços Web

- Organização das rotas
  - Agora que já organizamos nossos **pacotes** e **classes**, vamos organizar nossas **rotas**.
  - Neste primeiro projeto de API REST, temos apenas uma rota definida.



```
// Exemplo de endpoint usando Spring Boot
@GetMapping("/test")
public String helloWorld(){
    return service.welcomeMessage();
}
```

Construção de serviços Web

- Organização das rotas
  - A importação import org.springframework.web.bind.annotation.GetMapping; refere-se a uma anotação específica do Spring Framework usada para mapear métodos de controle (controllers) a endpoints HTTP específicos.

  - Essa anotação faz parte do módulo spring-web e é usada principalmente para criar APIs RESTful.

Construção de serviços Web

- Organização das rotas
  - Organizar rotas em uma API REST é uma prática importante para manter o código claro, **modular** e fácil de entender.

  - Existem várias abordagens para organizar rotas, e a escolha pode depender do tamanho e complexidade do seu projeto.

Construção de serviços Web

- Organização das rotas

  Aqui estão algumas estratégias comuns para organização de rotas:

  - **Organização por Recurso:**
    - Agrupe suas rotas com base nos recursos que a API fornece.
    - Cada recurso (por exemplo: usuários, produtos, pedidos) terá seu próprio conjunto de rotas. Isso pode ser feito usando um padrão como "/users", "/products", "/orders".
    - Por hora, estamos utilizando esta abordagem para organizar nossas rotas.

Construção de serviços Web

- Organização das rotas

  Aqui estão algumas estratégias comuns para organização de rotas:

  - **Organização por Funcionalidade:**
    - Agrupe suas rotas com base na funcionalidade que elas fornecem. Isso é útil quando você tem muitos recursos relacionados a uma funcionalidade específica.
    - Exemplo: Autenticação, faturas, relatórios, etc.

Construção de serviços Web

- Organização das rotas

  Aqui estão algumas estratégias comuns para organização de rotas:

  - **Organização por Versão da API:**
    - Se você espera que sua API evolua ao longo do tempo, pode ser útil versionar suas rotas (v1, v2, v3...) para garantir compatibilidade com versões anteriores.
      - /v1/users/
      - /v2/users/

Construção de serviços Web

- Organização das rotas

  Aqui estão algumas estratégias comuns para organização de rotas:

  - **Organização Hierárquica:**
    - Organize suas rotas de maneira hierárquica para representar relações de recursos aninhados.
      - /users
      - /users/{userId}
      - /users/{userId}/orders
      - /users/{userId}/orders/{orderId}

Construção de serviços Web

- Organização das rotas
  - Antes de avançarmos para o próximo exemplo prático, vamos criar mais uma rota de exemplo.

  - Teremos assim, duas rotas e dois serviços.

RestApiApplication.java | Service.java | Controller.java

```java
package com.example.RestAPI.controller;

import com.example.RestAPI.service.Service;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class Controller {

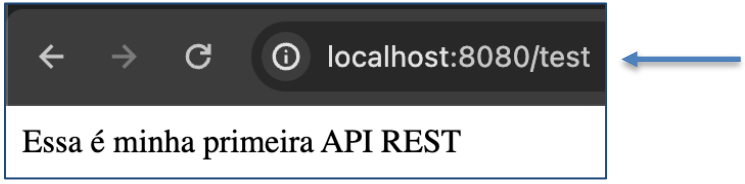
    //Classe de serviços
    2 usages
    Service service = new Service();

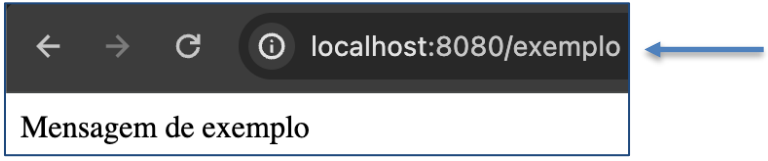
    // Exemplo de endpoint usando Spring Boot
    @GetMapping("/test")
    public String helloWorld(){
        return service.welcomeMessage();
    }

    @GetMapping("/exemplo")
    public String exemplo(){
        return service.exampleMessage();
    }
}
```

RestApiApplication.java | Service.java | Controller.java

```java
public class Service {
    1 usage
    public String welcomeMessage() {
        return "Essa é minha primeira API REST";
    }
    1 usage
    public String exampleMessage() {
        return "Mensagem de exemplo";
    }
}
```





Vamos à prática!

- Crie uma API REST para consultar o clima em BH.

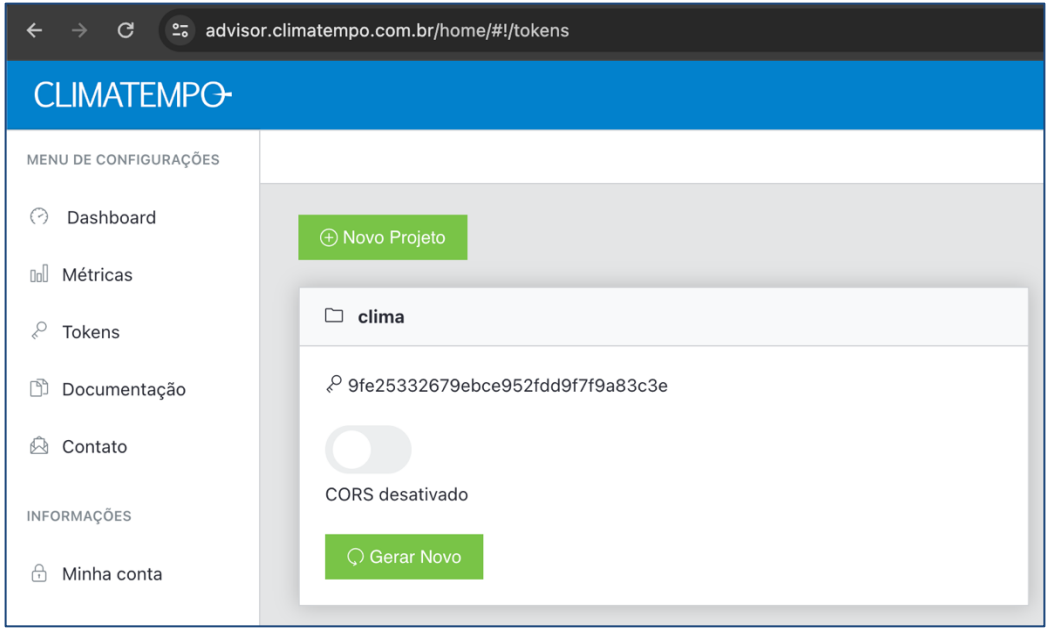
- Para consultar as informações sobre o clima, utilizaremos a API CLIMATEMPO.
  - Para verificar o clima de uma cidade apenas, a API é gratuita.
    - https://advisor.climatempo.com.br/

Vamos à prática!



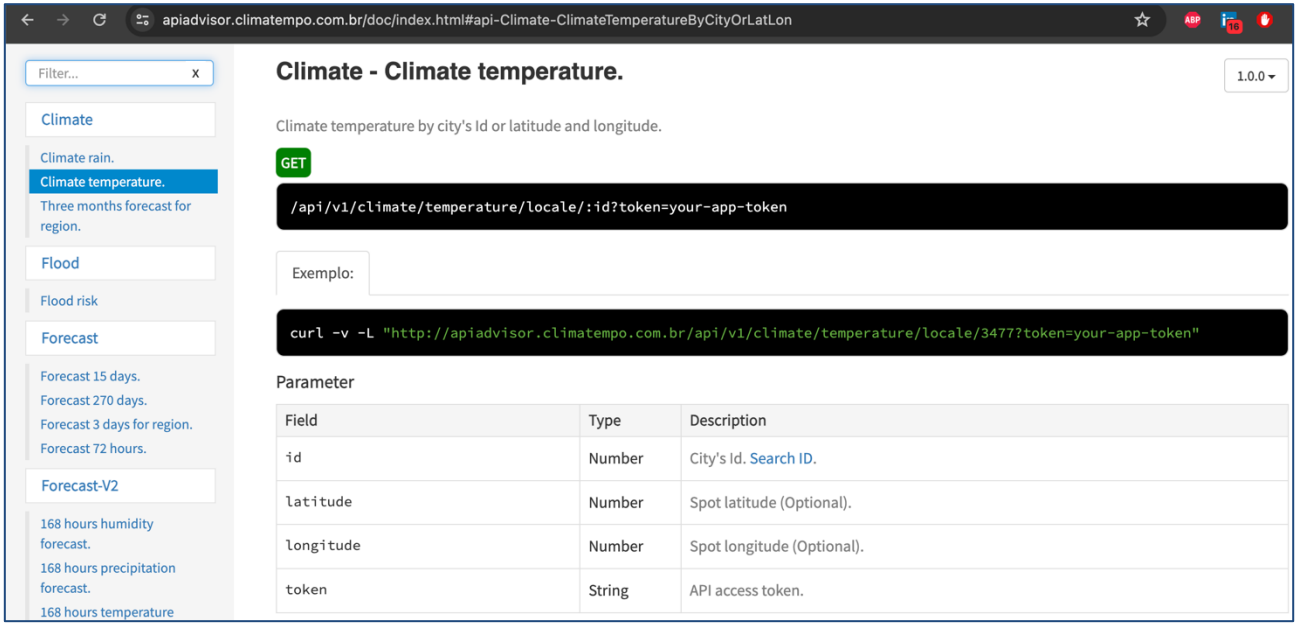
Vamos à prática!

- Para utilizar a API CLIMATEMPO é necessário cadastrar um email, logar e gerar um token:



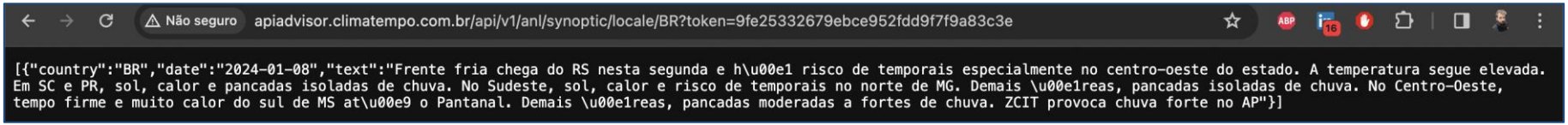
Vamos à prática!

- Link para a documentação que utilizaremos como guia:
- https://apiadvisor.climatempo.com.br/doc/index.html#api-Climate-ClimateTemperatureByCityOrLatLon



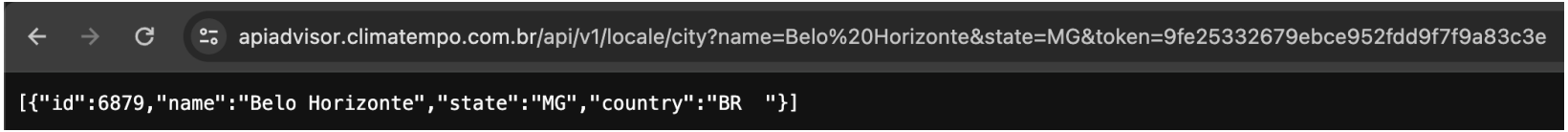
Vamos à prática!

- Antes de partir para o código, faça um teste no browser de sua preferência para verificar se a chamada à API está funcionando:
  - http://apiadvisor.climatempo.com.br/api/v1/anl/synoptic/locale/BR?token=9fe25332679ebce952fdd9f7f9a83c3e
  - Para o nível Brasil está funcionando corretamente.



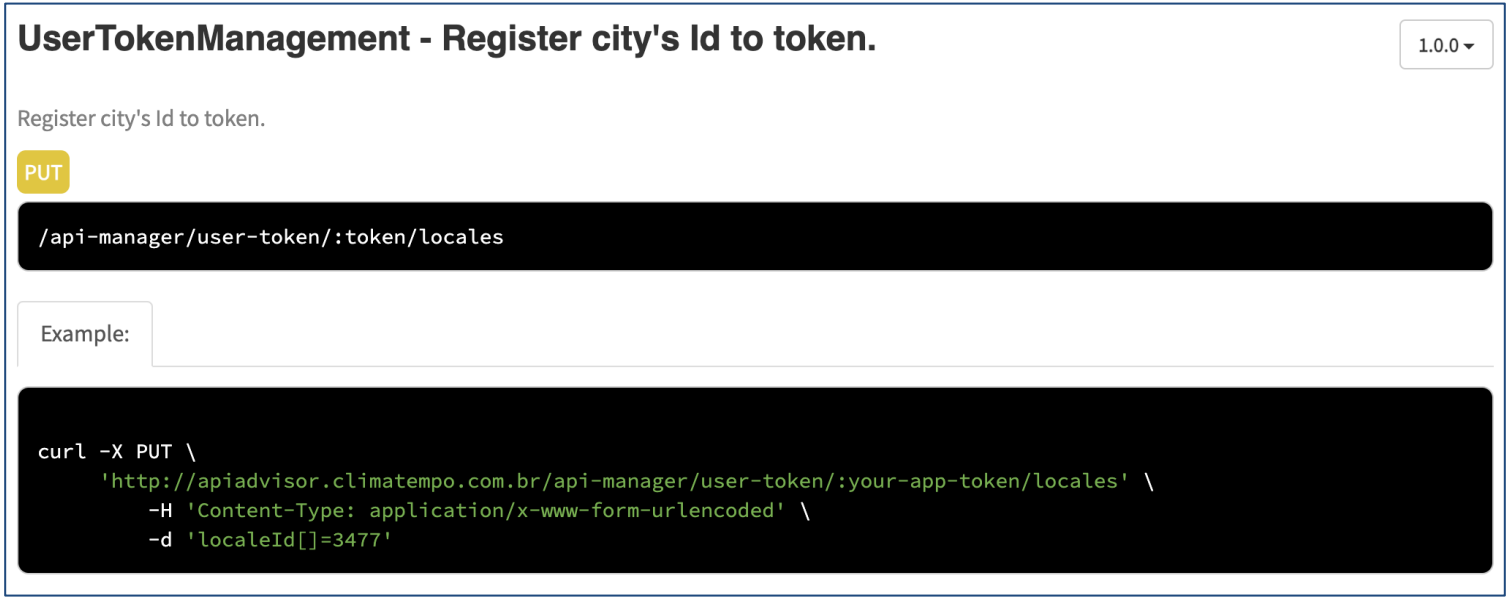
Vamos à prática!

- Já para o nível Belo Horizonte, será necessário cadastrar a cidade para aquele token gerado (lembrando que somente uma cidade é permitida no plano gratuito).
  - ID da cidade de Belo Horizonte: **6879**
    - https://apiadvisor.climatempo.com.br/api/v1/locale/city?name=Belo%20Horizonte&state=MG&token=9fe25332679ebce952fdd9f7f9a83c3e



Vamos à prática!

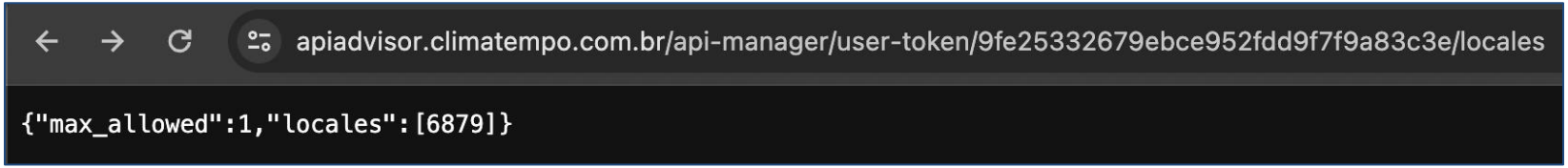
- Para referenciar o ID 6879 (BH) ao nosso token, será necessário rodar o seguinte comando:



Vamos à prática!

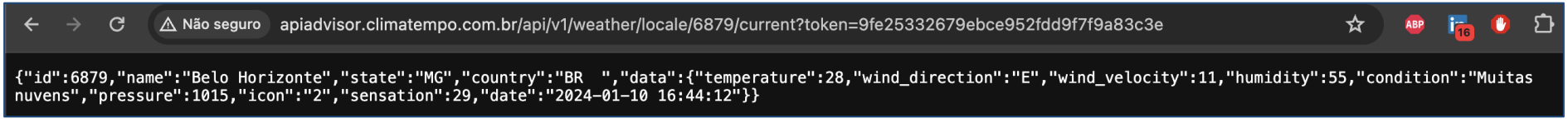
- Agora temos a cidade de BH cadastrada para o nosso token.
  - https://apiadvisor.climatempo.com.br/api-manager/user-token/9fe25332679ebce952fdd9f7f9a83c3e/locales

- Caso você não tenha interessa em criar um cadastro no CLIMATEMPO, ainda assim poderá utilizar as URLs desta aula normalmente.

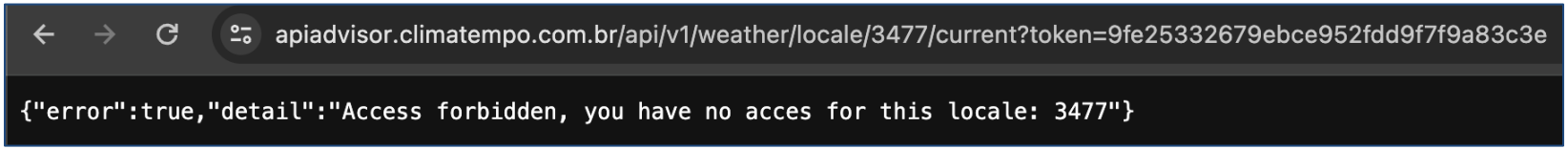


Vamos à prática!

- Agora temos as informações sobre o clima da cidade de Belo Horizonte aparecendo no browser (Google Chrome):
  - https://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e



- Caso não tivéssemos feito esse processo de registrar a cidade de BH para o token, a API retornaria a seguinte mensagem:



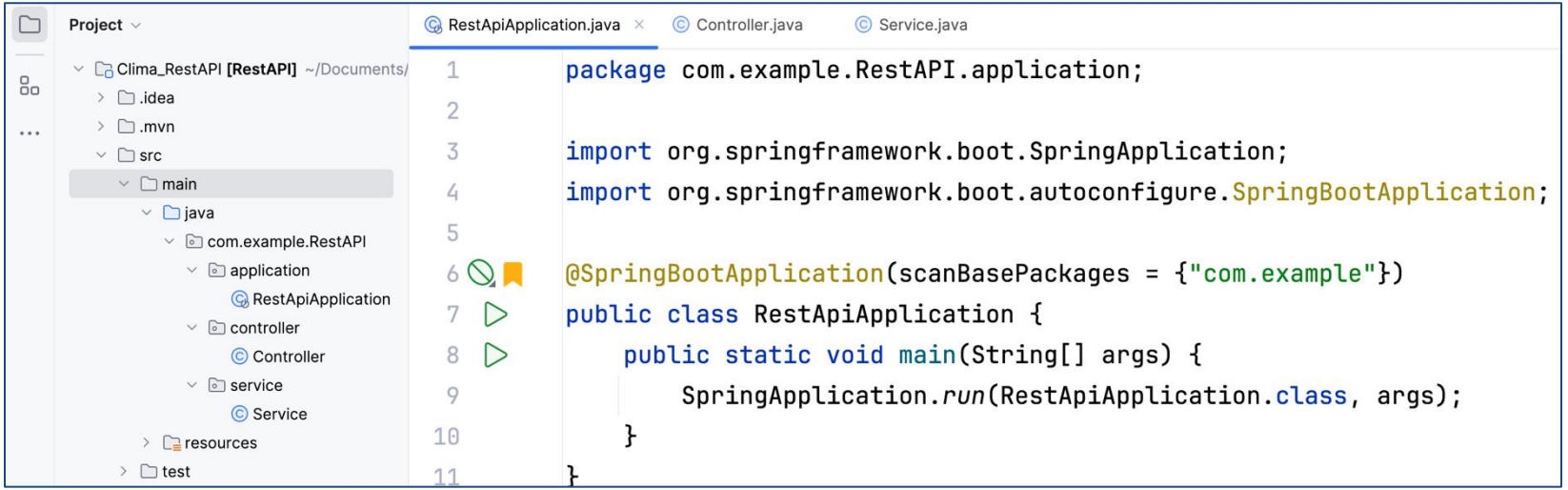
Vamos à prática!

- Uma vez que a chamada à API está funcionando no browser, podemos levar o link de BH para o nosso código Java.

- Faremos uma estrutura de projeto muito semelhante ao primeiro exemplo, com os pacotes **application**, **controller** e **service**.

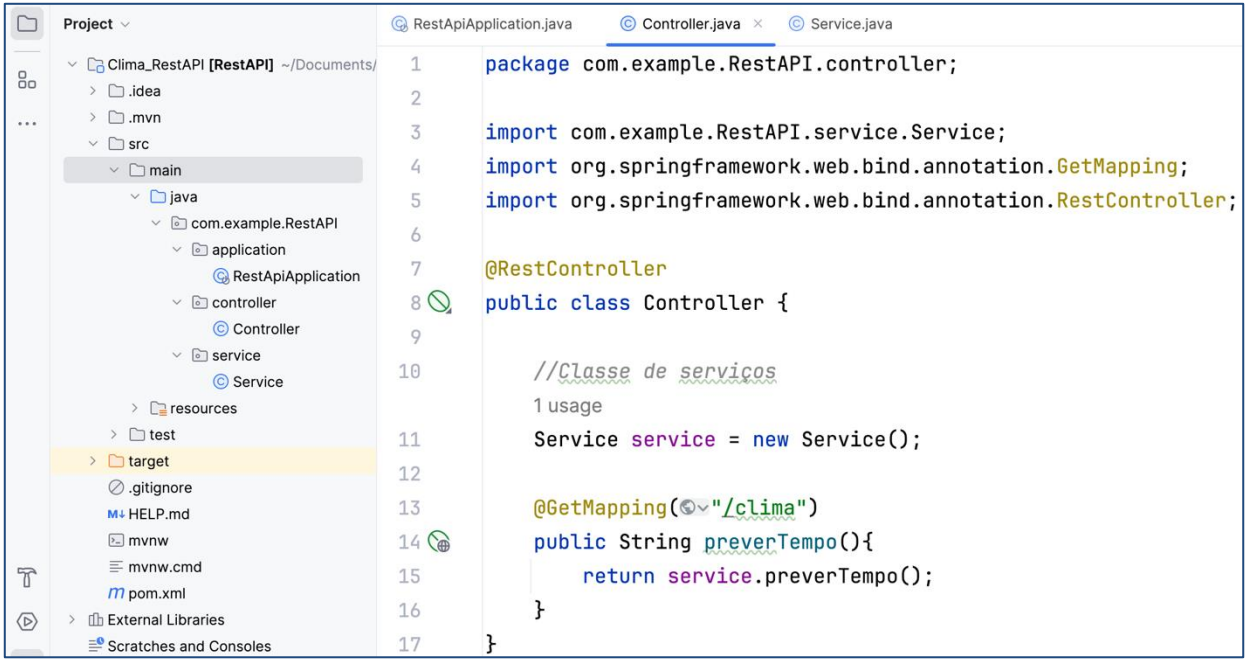
Vamos à prática!

- Classe **RestApiApplication** com o método **main**:



Vamos à prática!

- Classe **Controller** com o **endpoint** e a chamada para a classe **Service**:



- Classe **Service** contendo o serviço de previsão do tempo:
  - Repare que o link para o clima de BH foi inserido na variável String **apiUrl**.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Aqui temos a variável que será retornada do **Service** para o **Controller**.
  - String **dadosMeteorologicos**.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Aqui estamos utilizando a classe **RestTemplate** do **Spring** Framework para fazer uma requisição **HTTP GET** para uma determinada URL (**apiUrl**) e receber a resposta como uma String.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Aqui estamos verificando se a requisição foi bem-sucedida. Ou seja, se está na faixa de códigos 2xx (Normalmente 200 = sucesso, 500 = erro interno do servidor, 404 = página não encontrada, etc.).

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Caso a requisição tenha sido bem-sucedida, guardamos o corpo de sua resposta.
  - Utilizamos para isso o método **getBody()** da classe **ResponseEntity**.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Caso a requisição **não** tenha sido bem-sucedida, guardamos o código do status.
  - Utilizamos para isso o método **getStatusCode()** da classe **ResponseEntity**.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

- Por último, retornamos o resultado do nosso serviço, seja ele o corpo da resposta ou o código de status, para a classe **Controller**.

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

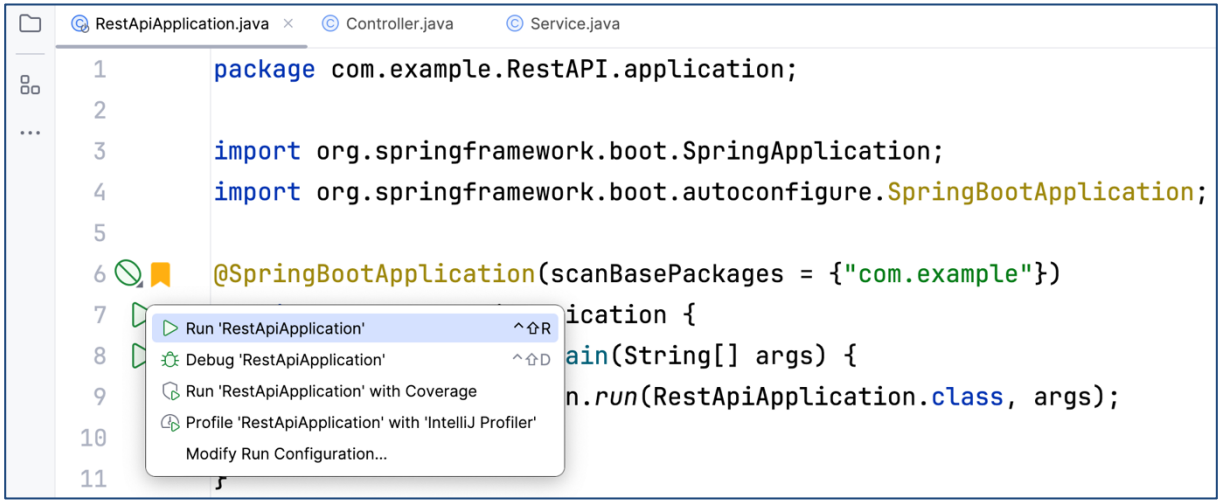
3 usages
public class Service {
    1 usage
    public String preverTempo() {
        String dadosMeteorologicos = "";
        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";

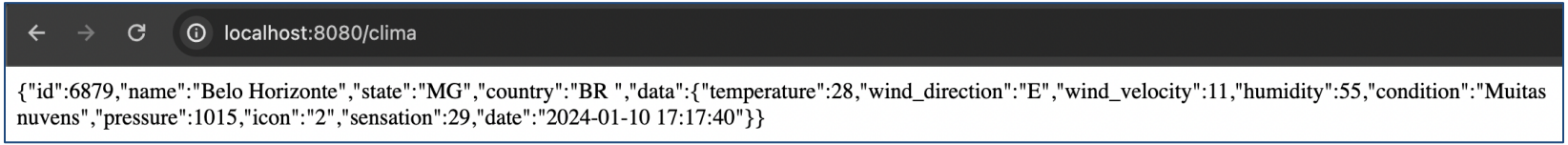
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);

        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dadosMeteorologicos = responseEntity.getBody();
        } else {
            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
        }
        return dadosMeteorologicos;
    }
}
```

Vamos à prática!

- Por fim, basta rodar a aplicação e ver o resultado, em formato **JSON**, no browser:





Bônus

- Até o momento, utilizamos o browser (**Google Chrome)** para enviar as requisições HTTP para a nossa API REST.
  - [GET] http://localhost:8080/clima

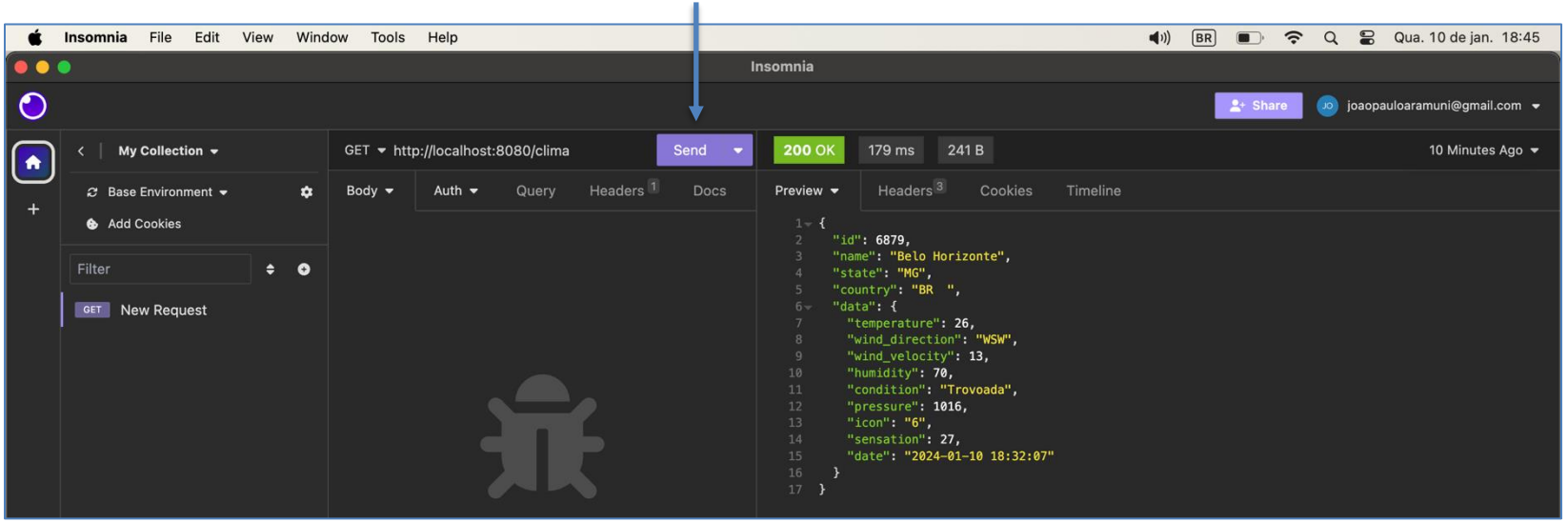
- Porém no mercado de trabalho é mais comum usarmos programas específicos para essa finalidade, como o **Postman** e o **Insomnia**.





Bônus

- Faça o download e teste você mesmo:
  - https://www.postman.com/downloads/
  - https://insomnia.rest/download



Exercício

- **FIPE API HTTP REST**
  - API de Consulta Tabela **FIPE**
- Crie uma **API REST** em Java, utilizando o Spring Framework, para consultar informações sobre marcas, modelos e valores de carros da tabela FIPE.
- Utilize a seguinte API para consumir as informações da tabela FIPE.
  - https://deividfortuna.github.io/fipe/
  - https://parallelum.com.br/fipe/api/v1



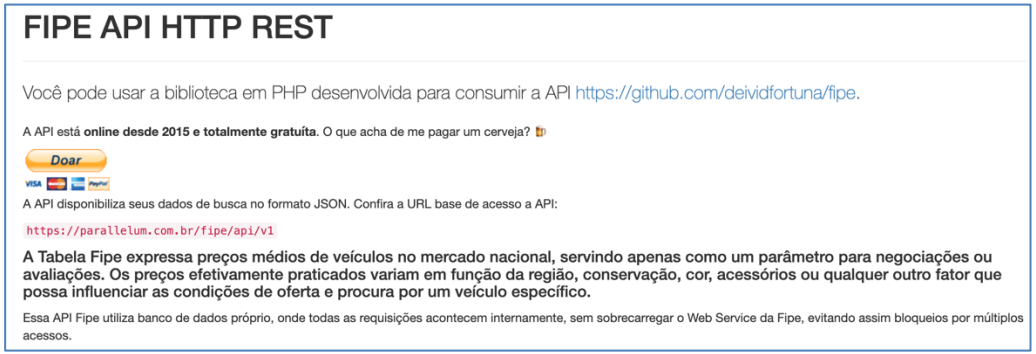
Exercício

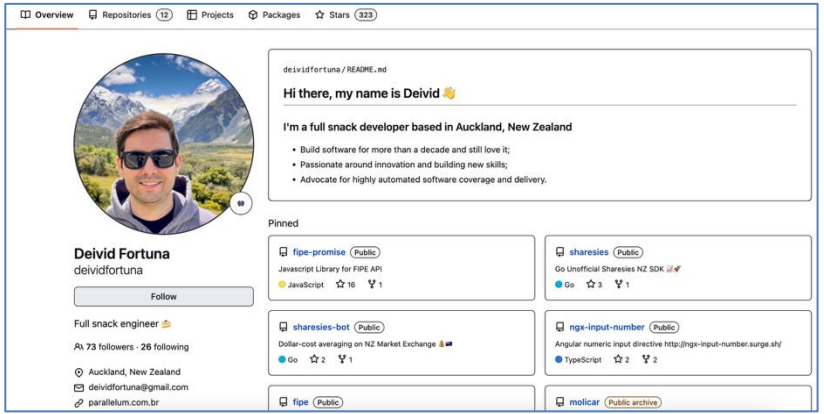
- **FIPE API HTTP REST**
  - A API de Consulta Tabela FIPE fornece preços médios de veículos no mercado nacional. Atualizada mensalmente com dados extraídos da tabela FIPE.

  - Essa API Fipe utiliza banco de dados próprio, onde todas as requisições acontecem internamente, sem sobrecarregar o Web Service da Fipe, evitando assim bloqueios por múltiplos acessos.

Exercício

- **FIPE API HTTP REST**
  - A API está online desde 2015 e é totalmente gratuita.
  - Ela é mantida pelo Dev Deivid Fortuna: deividfortuna@gmail.com
    - https://github.com/deividfortuna/fipe





Exercício

- **FIPE API HTTP REST**
  - Obs.: Existem outras APIs que também fornecem o mesmo serviço de consulta à Tabela FIPE, porém necessitam de cadastro, token e/ou são pagas.





Exercício

- **FIPE API HTTP REST**
  - Antes de partir para o código Java, faça um teste das **URLs** no browser de sua preferência:
    - https://parallelum.com.br/fipe/api/v1/carros/marcas



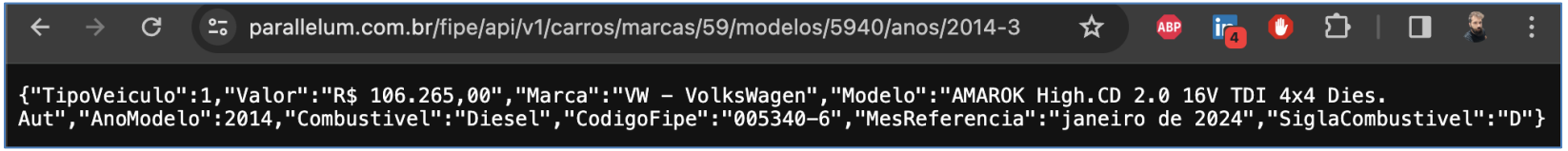
- Modelos da marca VW - VolksWagen (**59**):
  - https://parallelum.com.br/fipe/api/v1/carros/marcas/59/modelos



{"modelos":[{"codigo":5585,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x2 Die"},{"codigo":5586,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x4 Die"},{"codigo":9895,"nome":"AMAROK Comfor. 3.0 V6 TDI 4x4 Dies. Aut."},{"codigo":8531,"nome":"AMAROK Comfor. CD 2.0 TDI 4x4 Dies. Aut."},{"codigo":5748,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x2 Diesel"},
{"codigo":5749,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x4 Diesel"},{"codigo":8532,"nome":"AMAROK Extreme CD 3.0 4x4 TB Dies. Aut."},{"codigo":7854,"nome":"AMAROK Hig. Extreme CD 2.0 4x4 Dies. Aut"},{"codigo":7674,"nome":"AMAROK Hig.ULTIMATE CD 2.0 4x4 Dies. Aut"},{"codigo":5940,"nome":"AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut"},{"codigo":5261,"nome":"AMAROK Highline CD 2.0 16V TDI 4x4 Dies."},{"codigo":8183,"nome":"AMAROK Highline CD 3.0 4x4 TB Dies. Aut."},{"codigo":5476,"nome":"AMAROK SE CD 2.0 16V TDI 4x4 Diesel"},
{"codigo":7367,"nome":"AMAROK T. Dark Label CD 2.0 4x4 Dies Aut"},{"codigo":5350,"nome":"AMAROK Trendline CD 2.0 16V TDI 4x4 Dies"},{"codigo":6277,"nome":"AMAROK Trendline CD 2.0 TDI 4X4 Dies Aut"},{"codigo":2359,"nome":"Apolo GL 1.8"},{"codigo":2360,"nome":"Apolo GLS/ Vip 1.8"},{"codigo":2362,"nome":"Bora 2.0 8v Comfortline Aut."},
{"codigo":2363,"nome":"Bora 2.0 8v Comfortline Mec."},{"codigo":2361,"nome":"Bora 2.0/ 2.0 Flex 8v Aut."},{"codigo":2364,"nome":"Bora 2.0/ 2.0 Flex 8v Mec."},
{"codigo":2365,"nome":"Caravelle 2.4 Diesel"},{"codigo":2366,"nome":"Corrado 2.0 Turbo"},{"codigo":2367,"nome":"Corrado G-60 2.8"},{"codigo":7077,"nome":"CROSSFOX 1.6 T. Flex 16V 5p"},{"codigo":7078,"nome":"CROSSFOX I MOTION 1.6 T. Flex 16V 5p"},{"codigo":2368,"nome":"CROSSFOX 1.6 Mi Total Flex 8V 5p"},{"codigo":5983,"nome":"CROSSFOX I MOTION 1.6 Mi T. Flex 8V 5p"},{"codigo":10368,"nome":"Delivery Express Turbo Diesel (E6)"},{"codigo":10137,"nome":"Delivery Express+ Turbo Diesel"},{"codigo":4857,"nome":"EOS Cab. 2.0 Turbo FSI Tiptronic"},{"codigo":2369,"nome":"Eurovan 2.4 Diesel"},{"codigo":5082,"nome":"Fox 1.0 Mi Total Flex 8V 3p"},{"codigo":5083,"nome":"Fox 1.0 Mi Total Flex 8V 5p"},
{"codigo":5084,"nome":"Fox 1.6 Mi I MOTION Total Flex 8V 5p"},{"codigo":5085,"nome":"Fox 1.6 Mi Total Flex 8V 5p"},{"codigo":6548,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 3p"},
{"codigo":6549,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 5p"},{"codigo":6039,"nome":"Fox BlueMotion 1.6 Mi Total Flex 3p."},{"codigo":6040,"nome":"Fox BlueMotion 1.6 Mi Total Flex 5p."},{"codigo":2370,"nome":"Fox City 1.0 Mi/ 1.0Mi Total Flex 8V 5p"},{"codigo":2371,"nome":"Fox City 1.0Mi/ 1.0Mi Total Flex 8V 3p"},{"codigo":7269,"nome":"Fox Comfortline 1.0 Flex 12V 5p"},{"codigo":6934,"nome":"Fox Comfortline 1.0 Flex 8V 5p"},{"codigo":6935,"nome":"Fox Comfortline 1.6 Flex 8V 5p"},{"codigo":6936,"nome":"Fox Comfortline I Motion 1.6 Flex 8V 5p"},{"codigo":8112,"nome":"Fox Connect 1.6 Flex 8V 5p"},{"codigo":8133,"nome":"Fox Connect I Motion 1.6 Flex 8V 5p"},{"codigo":4668,"nome":"Fox extreme 1.6 Mi Total Flex 8V 5p"},{"codigo":6937,"nome":"Fox Highline I MOTION 1.6 Flex 16V 5p"},{"codigo":6938,"nome":"Fox Highline1.6 Flex 16V 5p"},{"codigo":7172,"nome":"Fox PEPPER 1.6 Flex 16V 5p"},
{"codigo":7173,"nome":"Fox PEPPER I MOTION 1.6 Flex 16V 5p"},{"codigo":2372,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 3p"},{"codigo":2373,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 4p"},{"codigo":2374,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 3p"},{"codigo":2375,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 4p"},{"codigo":5087,"nome":"Fox PRIME/Hghi. IMOTION 1.6 T.Flex 8V 5p"},{"codigo":5086,"nome":"Fox PRIME/Higli. 1.6 Total Flex 8V 5p"},{"codigo":5599,"nome":"Fox Rock in Rio 1.6 Mi Total Flex 8V 5p"},{"codigo":4349,"nome":"Fox Route 1.0 Mi Total Flex 8V 3p"},{"codigo":4350,"nome":"Fox Route 1.0 Mi Total Flex 8V 5p"},{"codigo":4351,"nome":"Fox Route 1.6 Mi Total Flex 8V 3p"},{"codigo":4352,"nome":"Fox Route 1.6 Mi Total Flex 8V 5p"},{"codigo":7637,"nome":"Fox RUN 1.6 Flex 8V 5p"},{"codigo":6667,"nome":"Fox SELEÇÃO 1.0 Total Flex 8V 5p"},{"codigo":6668,"nome":"Fox SELEÇÃO 1.6 Total Flex 8V 5p"},{"codigo":6784,"nome":"Fox SELEÇÃO IMOTION 1.6 Mi T. Flex 8V 5p"},{"codigo":2379,"nome":"Fox Sportline/Sports 1.0 Tot.Flex 8V 4p"},{"codigo":2376,"nome":"Fox Sportline/Sports 1.0/1.0 Tot.Flex 3p"},{"codigo":2377,"nome":"Fox Sportline/Sports 1.6/1.6 Tot.Flex 3p"},{"codigo":2378,"nome":"Fox Sportline/Sports 1.6/1.6 Tot.Flex 4p"},
{"codigo":4928,"nome":"Fox SUNRISE 1.0 Mi Total Flex 8V 5p"},{"codigo":7410,"nome":"Fox TRACK 1.0 Flex 12V 5p"},{"codigo":7295,"nome":"Fox Trendline 1.0 Flex 12V 5p"},
{"codigo":6939,"nome":"Fox Trendline 1.0 Flex 8V 5p"},{"codigo":6940,"nome":"Fox Trendline 1.6 Flex 8V 5p"},{"codigo":8113,"nome":"Fox Xtreme 1.6 Flex 8V 5p"},
{"codigo":2380,"nome":"Fusca"},{"codigo":7079,"nome":"Fusca 2.0 R-Line TSI 16V Aut."},{"codigo":6278,"nome":"Fusca 2.0 TSI 16V Aut."},{"codigo":6279,"nome":"Fusca 2.0 TSI 16V Mec."},{"codigo":6280,"nome":"Gol (novo) 1.0 Mi Total Flex 8V 2p"},{"codigo":4689,"nome":"Gol (novo) 1.0 Mi Total Flex 8V 4p"},{"codigo":6281,"nome":"Gol (novo) 1.6 Mi Total Flex 8V 2p"},{"codigo":4691,"nome":"Gol (novo) 1.6 Mi Total Flex 8V 4p"},{"codigo":4690,"nome":"Gol (novo) 1.6 Power/Highi T.Flex 8V 4P"},{"codigo":8323,"nome":"Gol 1.0 Flex 12V 5p"},
{"codigo":2381,"nome":"Gol 1.0 Mi FUN/ Highway/ Sport 16V 2/4p"},{"codigo":2382,"nome":"Gol 1.0 Plus 16v 2p"},{"codigo":2383,"nome":"Gol 1.0 Plus 16v 4p"},
{"codigo":2384,"nome":"Gol 1.0 Plus 8v 2p"},{"codigo":2385,"nome":"Gol 1.0 Plus 8v 4p"},{"codigo":2386,"nome":"Gol 1.0 Power 16v 76cv 4p"},{"codigo":5941,"nome":"Gol 1.0 Total Flex 8V 5p (25 Anos)"},{"codigo":2387,"nome":"Gol 1.0 Trend/ Power 8V 2p"},{"codigo":2388,"nome":"Gol 1.0 Trend/ Power 8V 4p"},{"codigo":5114,"nome":"Gol 1.6 I MOTI.Power/Highli T.Flex 8V 4p"},{"codigo":6454,"nome":"Gol 1.6 Mi I MOTION Total Flex 8V 3p"},{"codigo":2389,"nome":"Gol 1.6 Mi Plus Total Flex 8V 2p"},{"codigo":2390,"nome":"Gol 1.6 Mi Plus Total Flex 8V 4p"},{"codigo":2391,"nome":"Gol 1.6 Mi Power Total Flex 8V 4p"},{"codigo":5404,"nome":"Gol 1.6 Mi Rallye I MOTION T. Flex 8V 4p"},{"codigo":2392,"nome":"Gol 1.6 Mi Rallye Total Flex 8V 4p"},{"codigo":2393,"nome":"Gol 1.6 Mi/ 1.6i 2p"},{"codigo":2394,"nome":"Gol 1.6 Mi/ Power 1.6 Mi 4p"},{"codigo":8463,"nome":"Gol 1.6 MSI Flex 16V 5p Aut."},
{"codigo":8324,"nome":"Gol 1.6 MSI Flex 8V 5p"},{"codigo":2395,"nome":"Gol 1.8 Mi"},{"codigo":2396,"nome":"Gol 1.8 Mi 4p"},{"codigo":2397,"nome":"Gol 1.8 Mi Power Total Flex 8V 4p"},{"codigo":2398,"nome":"Gol 1.8 Mi Rallye Total Flex 8V 4p"},{"codigo":2399,"nome":"Gol 1000 (modelo antigo)"},{"codigo":2400,"nome":"Gol 1000 Mi 16V 2p Turbo"},
{"codigo":2401,"nome":"Gol 1000 Mi 16V 4p Turbo"},{"codigo":2402,"nome":"Gol 1000 Mi 16V/ Ouro 2p"},{"codigo":2403,"nome":"Gol 1000 Mi 16V/ Ouro 4p"},{"codigo":2404,"nome":"Gol 1000 Mi 2p / 1000i"},{"codigo":2405,"nome":"Gol 1000 Mi 4p"},{"codigo":2406,"nome":"Gol 1000 Mi Plus 16V 2p e 4p"},{"codigo":2407,"nome":"Gol 1000 Mi Plus 8v 2p e 4p"},
{"codigo":2408,"nome":"Gol 1000i Plus 2p"},{"codigo":2409,"nome":"Gol 2.0 Mi 2p"},{"codigo":2410,"nome":"Gol 2.0 Mi 4p"},{"codigo":5750,"nome":"Gol BLACK 1.0 Mi Total Flex 8V 4p"},
{"codigo":2413,"nome":"Gol City (Trend) 1.0 Mi Total Flex 8V 2p"},{"codigo":3832,"nome":"Gol City (Trend) 1.6 Mi T. Flex 8V 2p"},{"codigo":2417,"nome":"Gol City (Trend) 1.6 Mi T.Flex 8V 4p"},{"codigo":2414,"nome":"Gol City (Trend)/Titan 1.0 T. Flex 8V 4p"},{"codigo":2411,"nome":"Gol City 1.0 Mi 8V 2p"},{"codigo":2412,"nome":"Gol City 1.0 Mi 8V 4p"},
{"codigo":9098,"nome":"Gol City 1.0 Total Flex 12V 2p"},{"codigo":8840,"nome":"Gol City 1.0 Total Flex 12V 4P"},{"codigo":2415,"nome":"Gol City 1.6 Mi 8V 2p"},
{"codigo":2416,"nome":"Gol City 1.6 Mi 8V 4p"},{"codigo":2418,"nome":"Gol CL 1.6 Mi 2p e 4p"},{"codigo":2419,"nome":"Gol CL 1.8 Mi 2p e 4p"},{"codigo":2420,"nome":"Gol CLi / CL 1.8"},{"codigo":2421,"nome":"Gol CLi / CL/ Copa/ Stones 1.6"},{"codigo":6791,"nome":"Gol Comfort. I MOTION 1.6 T. Flex 8V 3p"},{"codigo":6792,"nome":"Gol Comfort. I MOTION 1.6 T. Flex 8V 5p"},{"codigo":7521,"nome":"Gol Comfortline 1.0 T. Flex 12V 5p"},{"codigo":6787,"nome":"Gol Comfortline 1.0 T. Flex 8V 3p"},{"codigo":6788,"nome":"Gol Comfortline 1.0 T.
Flex 8V 5p"},{"codigo":6789,"nome":"Gol Comfortline 1.6 T. Flex 8V 3p"},{"codigo":6790,"nome":"Gol Comfortline 1.6 T. Flex 8V 5p"},{"codigo":3958,"nome":"Gol COPA 1.0 Mi Total Flex

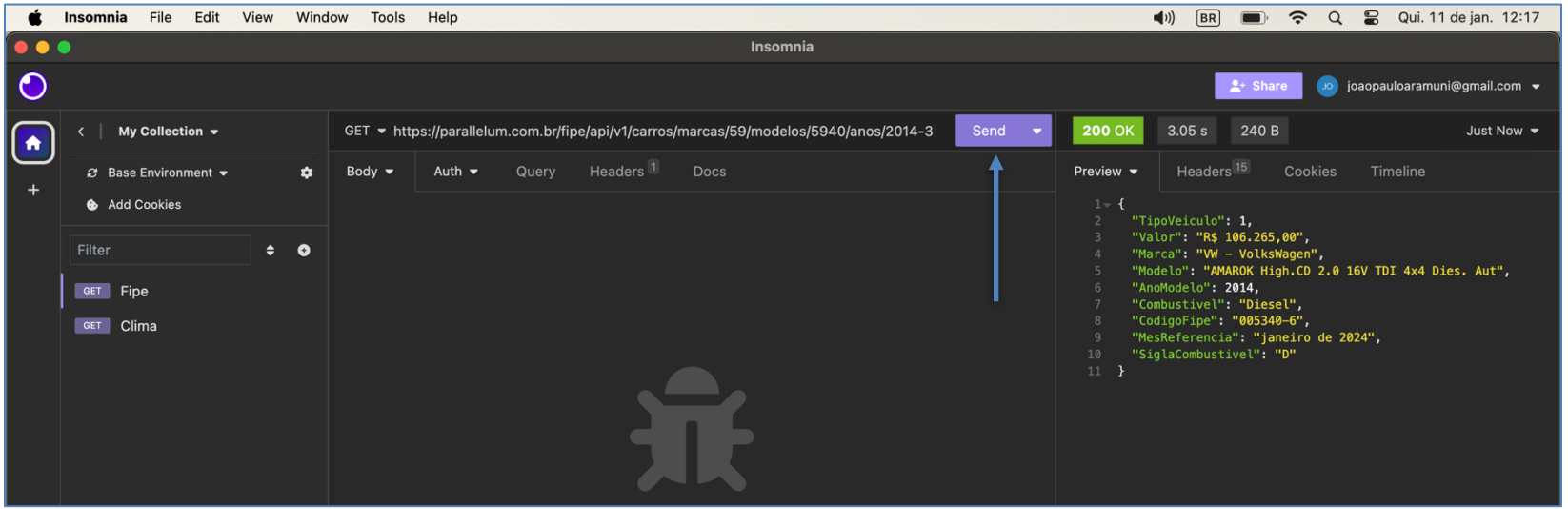
Exercício

- **FIPE API HTTP REST**
  - Por fim, o preço corrente da Tabela FIPE para um veículo/modelo/ano.
  - Valor de uma AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut ano 2014 a Diesel:
    - https://parallelum.com.br/fipe/api/v1/carros/marcas/59/modelos/5940/anos/2014-3
    - **R$ 106.265,00**



Exercício

- Faça o teste da URL também em uma plataforma de APIs, como Postman e Insomnia:



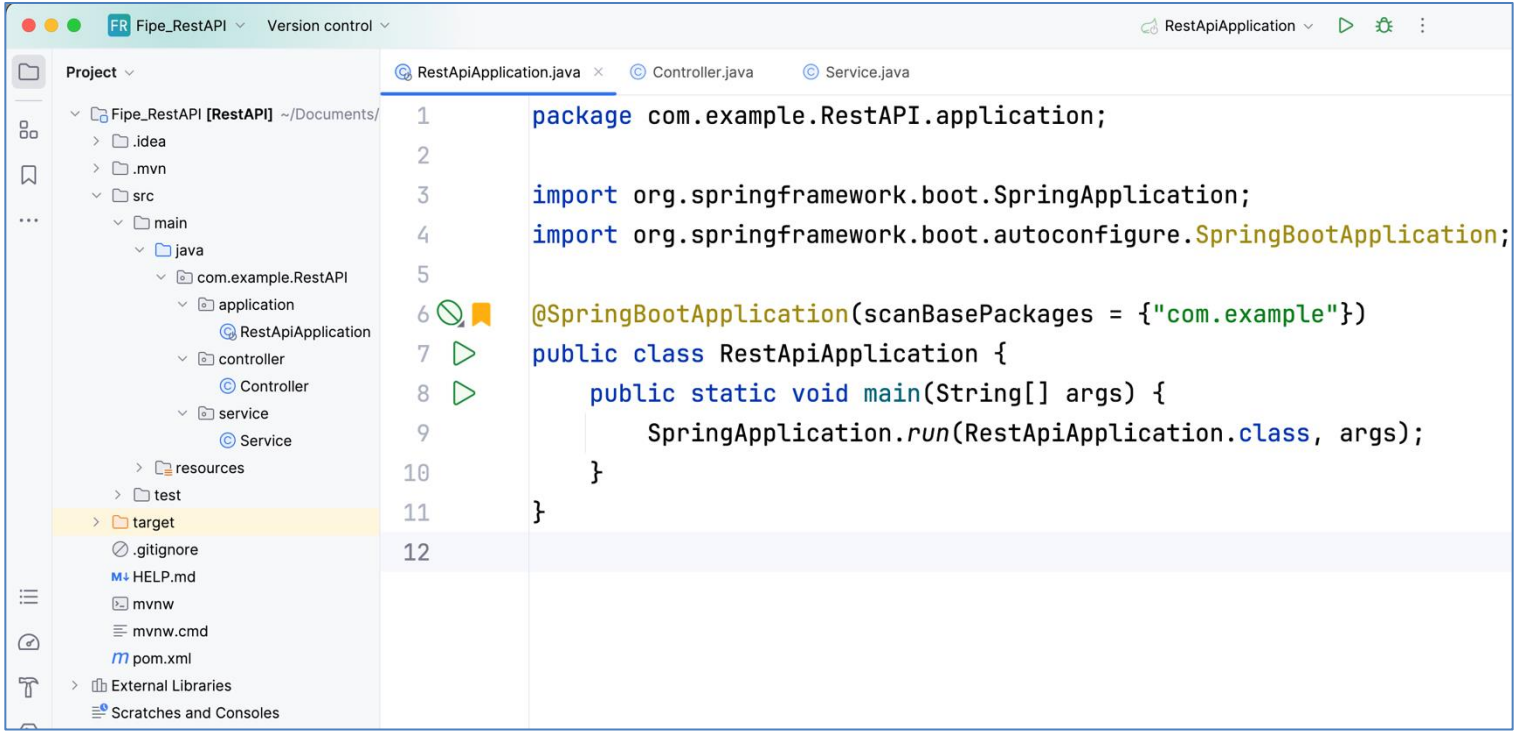
Exercício

- Uma vez que a chamada à API está funcionando no browser, podemos levar os links/URLs para o nosso código Java.

- Faremos uma estrutura de projeto muito semelhante ao exemplo anterior, com os pacotes **application**, **controller** e **service**.

Exercício

- Classe **RestApiApplication** com o método **main**:



Classe **Controller** com os **endpoints** e as chamadas para a classe **Service**:

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.controller;
import com.example.RestAPI.service.Service;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class Controller {
    Service service = new Service();

    @GetMapping("/marcas")
    public String consultarMarcas(){
        return service.consultarMarcas();
    }
    @GetMapping("/modelos/{marca}")
    public String consultarModelos(@PathVariable int marca){
        return service.consultarModelos(marca);
    }
    @GetMapping("/anos/{marca}/{modelo}")
    public String consultarAnos(@PathVariable int marca, @PathVariable int modelo){
        return service.consultarAnos(marca, modelo);
    }
    @GetMapping("/valor/{marca}/{modelo}/{ano}")
    public String consultarValor(@PathVariable int marca, @PathVariable int modelo, @PathVariable String ano){
        return service.consultarValor(marca, modelo, ano);
    }
}
```

Repare que agora estamos passando parâmetros na URL por meio da annotation **@PathVariable**

Temos 4 endpoints e consequentemente 4 rotas

Classe **Service** contendo os serviços de consulta de marcas, modelos, anos e valor:

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

public class Service {

    private String consultarURL(String apiUrl){
        String dados = "";
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dados = responseEntity.getBody();
        } else {
            dados = "Falha ao obter dados. Código de status: " + responseEntity.getStatusCode();
        }
        return dados;
    }
    public String consultarMarcas() {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas");
    }
    public String consultarModelos(int id) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+id+"/modelos");
    }
    public String consultarAnos(int marca, int modelo) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos");
    }
    public String consultarValor(int marca, int modelo, String ano) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos/"+ano);
    }
}
```

Extraindo a lógica para não duplicar código em todos os métodos

Classe **Service** contendo os serviços de consulta de marcas, modelos, anos e valor:

RestApiApplication.java | Controller.java | Service.java

```java
package com.example.RestAPI.service;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

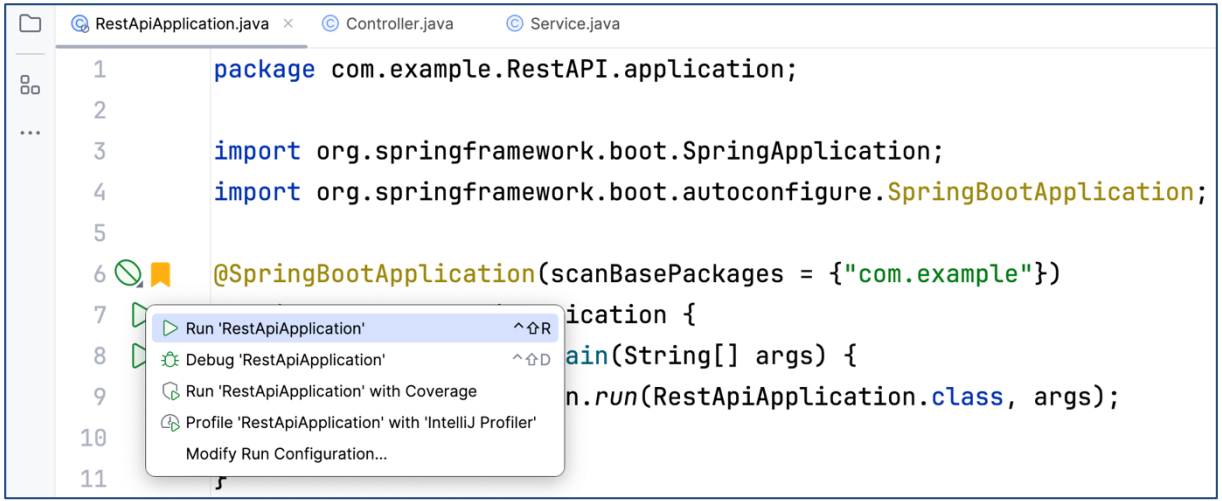
public class Service {

    private String consultarURL(String apiUrl){
        String dados = "";
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
        if (responseEntity.getStatusCode().is2xxSuccessful()) {
            dados = responseEntity.getBody();
        } else {
            dados = "Falha ao obter dados. Código de status: " + responseEntity.getStatusCode();
        }
        return dados;
    }
    public String consultarMarcas() {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas");
    }
    public String consultarModelos(int id) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+id+"/modelos");
    }
    public String consultarAnos(int marca, int modelo) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos");
    }
    public String consultarValor(int marca, int modelo, String ano) {
        return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos/"+ano);
    }
}
```

Serviços/regras de negócio separados do controller

Exercício

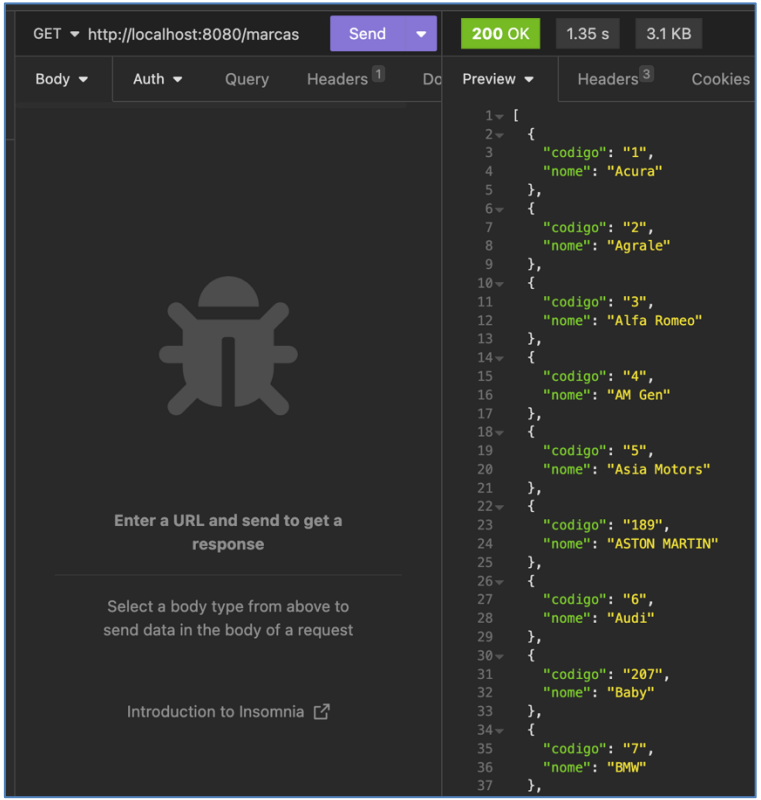
- Por fim, basta rodar a aplicação e ver o resultado, em formato **JSON**, no browser ou no Postman/Insomnia:



Marcas

localhost:8080/marcas

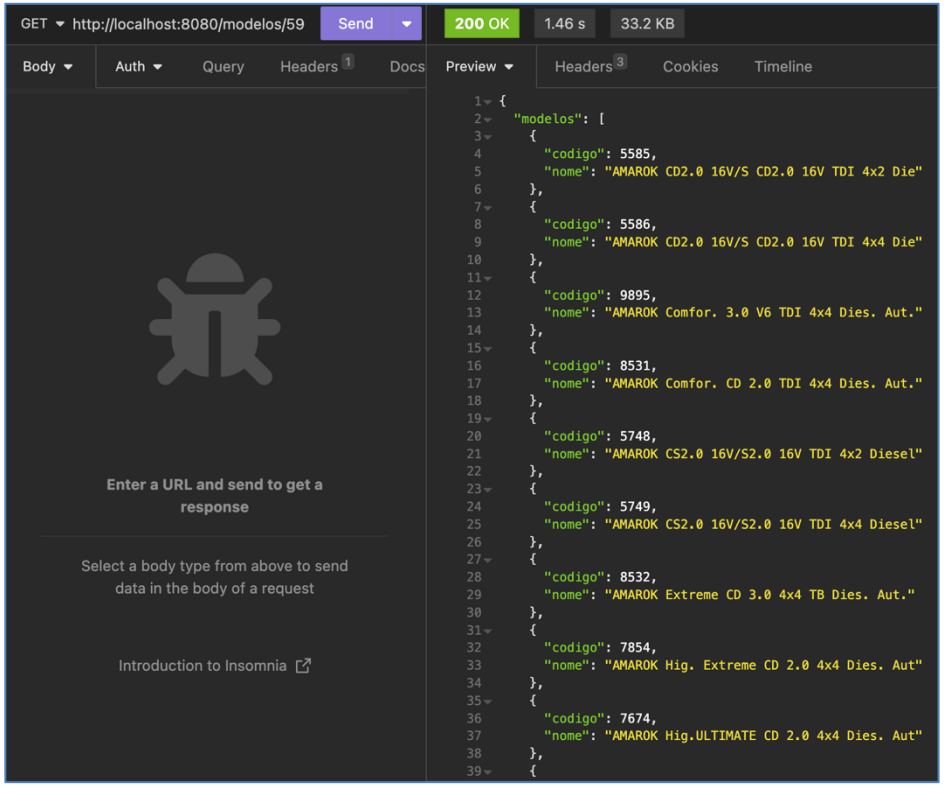
[{"codigo":"1","nome":"Acura"},{"codigo":"2","nome":"Agrale"},{"codigo":"3","nome":"Alfa Romeo"},
{"codigo":"4","nome":"AM Gen"},{"codigo":"5","nome":"Asia Motors"},{"codigo":"189","nome":"ASTON MARTIN"},
{"codigo":"6","nome":"Audi"},{"codigo":"207","nome":"Baby"},{"codigo":"7","nome":"BMW"},
{"codigo":"8","nome":"BRM"},{"codigo":"123","nome":"Bugre"},{"codigo":"238","nome":"BYD"},
{"codigo":"236","nome":"CAB Motors"},{"codigo":"10","nome":"Cadillac"},{"codigo":"245","nome":"Caoa Chery"},
{"codigo":"161","nome":"Caoa Chery/Chery"},{"codigo":"11","nome":"CBT Jipe"},{"codigo":"136","nome":"CHANA"},
{"codigo":"182","nome":"CHANGAN"},{"codigo":"12","nome":"Chrysler"},{"codigo":"13","nome":"Citroën"},
{"codigo":"14","nome":"Cross Lander"},{"codigo":"241","nome":"D2D Motors"},{"codigo":"15","nome":"Daewoo"},
{"codigo":"16","nome":"Daihatsu"},{"codigo":"246","nome":"DFSK"},{"codigo":"17","nome":"Dodge"},
{"codigo":"147","nome":"EFFA"},{"codigo":"18","nome":"Engesa"},{"codigo":"19","nome":"Envemo"},
{"codigo":"20","nome":"Ferrari"},{"codigo":"21","nome":"Fiat"},{"codigo":"149","nome":"Fibravan"},
{"codigo":"22","nome":"Ford"},{"codigo":"190","nome":"FOTON"},{"codigo":"170","nome":"Fyber"},
{"codigo":"199","nome":"GEELY"},{"codigo":"23","nome":"GM - Chevrolet"},{"codigo":"153","nome":"GREAT
WALL"},{"codigo":"24","nome":"Gurgel"},{"codigo":"240","nome":"GWM"},{"codigo":"152","nome":"HAFEI"},
{"codigo":"214","nome":"HITECH ELECTRIC"},{"codigo":"25","nome":"Honda"},{"codigo":"26","nome":"Hyundai"},
{"codigo":"27","nome":"Isuzu"},{"codigo":"208","nome":"IVECO"},{"codigo":"177","nome":"JAC"},
{"codigo":"28","nome":"Jaguar"},{"codigo":"29","nome":"Jeep"},{"codigo":"154","nome":"JINBEI"},
{"codigo":"30","nome":"JPX"},{"codigo":"31","nome":"Kia Motors"},{"codigo":"32","nome":"Lada"},
{"codigo":"171","nome":"LAMBORGHINI"},{"codigo":"33","nome":"Land Rover"},{"codigo":"34","nome":"Lexus"},
{"codigo":"168","nome":"LIFAN"},{"codigo":"127","nome":"LOBINI"},{"codigo":"35","nome":"Lotus"},
{"codigo":"140","nome":"Mahindra"},{"codigo":"36","nome":"Maserati"},{"codigo":"37","nome":"Matra"},
{"codigo":"38","nome":"Mazda"},{"codigo":"211","nome":"Mclaren"},{"codigo":"39","nome":"Mercedes-Benz"},
{"codigo":"40","nome":"Mercury"},{"codigo":"167","nome":"MG"},{"codigo":"156","nome":"MINI"},
{"codigo":"41","nome":"Mitsubishi"},{"codigo":"42","nome":"Miura"},{"codigo":"43","nome":"Nissan"},
{"codigo":"44","nome":"Peugeot"},{"codigo":"45","nome":"Plymouth"},{"codigo":"46","nome":"Pontiac"},
{"codigo":"47","nome":"Porsche"},{"codigo":"185","nome":"RAM"},{"codigo":"186","nome":"RELY"},
{"codigo":"48","nome":"Renault"},{"codigo":"195","nome":"Rolls-Royce"},{"codigo":"49","nome":"Rover"},
{"codigo":"50","nome":"Saab"},{"codigo":"51","nome":"Saturn"},{"codigo":"52","nome":"Seat"},
{"codigo":"247","nome":"SERES"},{"codigo":"183","nome":"SHINERAY"},{"codigo":"157","nome":"smart"},
{"codigo":"125","nome":"SSANGYONG"},{"codigo":"54","nome":"Subaru"},{"codigo":"55","nome":"Suzuki"},
{"codigo":"165","nome":"TAC"},{"codigo":"56","nome":"Toyota"},{"codigo":"57","nome":"Troller"},
{"codigo":"58","nome":"Volvo"},{"codigo":"59","nome":"VW - VolksWagen"},{"codigo":"163","nome":"Wake"},
{"codigo":"120","nome":"Walk"}]



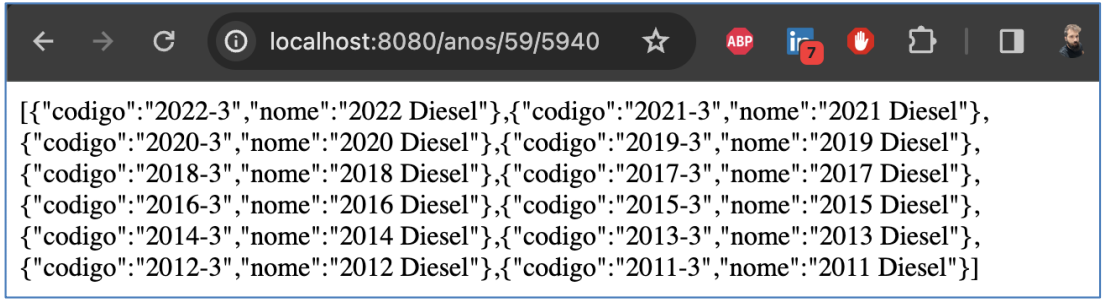
Modelos

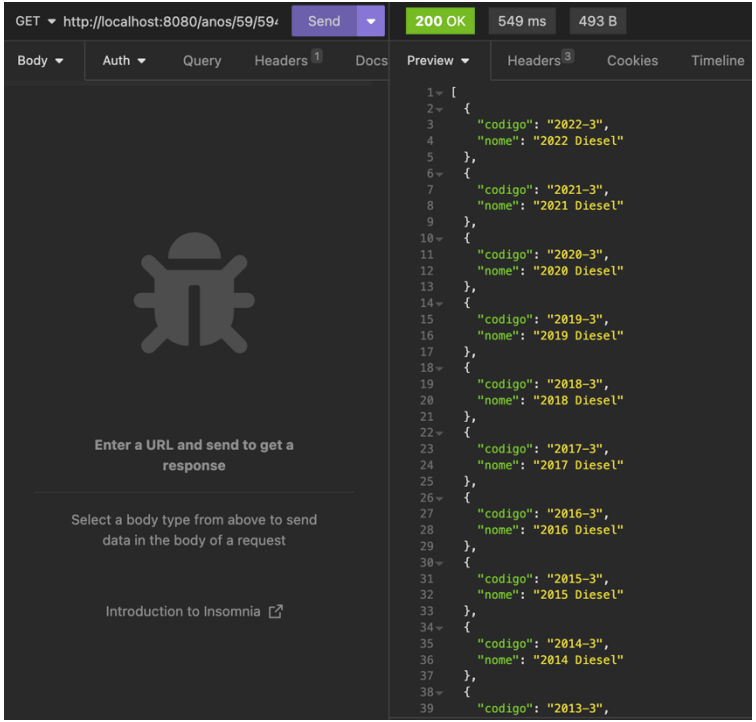
localhost:8080/modelos/59

{"modelos":[{"codigo":5585,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x2 Die"},
{"codigo":5586,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x4 Die"},{"codigo":9895,"nome":"AMAROK Comfor. 3.0 V6 TDI 4x4 Dies. Aut."},{"codigo":8531,"nome":"AMAROK Comfor. CD 2.0 TDI 4x4 Dies. Aut."},
{"codigo":5748,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x2 Diesel"},{"codigo":5749,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x4 Diesel"},{"codigo":8532,"nome":"AMAROK Extreme CD 3.0 4x4 TB Dies. Aut."},
{"codigo":7854,"nome":"AMAROK Hig. Extreme CD 2.0 4x4 Dies. Aut"},{"codigo":7674,"nome":"AMAROK Hig.ULTIMATE CD 2.0 4x4 Dies. Aut"},{"codigo":5940,"nome":"AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut"},
{"codigo":5261,"nome":"AMAROK Highline CD 2.0 16V TDI 4x4 Dies."},{"codigo":8183,"nome":"AMAROK Highline CD 3.0 4x4 TB Dies. Aut."},{"codigo":5476,"nome":"AMAROK SE CD 2.0 16V TDI 4x4 Diesel"},
{"codigo":7367,"nome":"AMAROK T. Dark Label CD 2.0 4x4 Dies Aut"},{"codigo":5350,"nome":"AMAROK Trendline CD 2.0 16V TDI 4x4 Dies"},{"codigo":6277,"nome":"AMAROK Trendline CD 2.0 TDI 4X4 Dies Aut"},
{"codigo":2359,"nome":"Apolo GL 1.8"},{"codigo":2360,"nome":"Apolo GLS/ Vip 1.8"},
{"codigo":2362,"nome":"Bora 2.0 8v Comfortline Aut."},{"codigo":2363,"nome":"Bora 2.0 8v Comfortline Mec."},
{"codigo":2361,"nome":"Bora 2.0/ 2.0 Flex 8v Aut."},{"codigo":2364,"nome":"Bora 2.0/ 2.0 Flex 8v Mec."},
{"codigo":2365,"nome":"Caravelle 2.4 Diesel"},{"codigo":2366,"nome":"Corrado 2.0 Turbo"},
{"codigo":2367,"nome":"Corrado G-60 2.8"},{"codigo":7077,"nome":"CROSSFOX 1.6 T. Flex 16V 5p"},
{"codigo":7078,"nome":"CROSSFOX I MOTION 1.6 T. Flex 16V 5p"},{"codigo":2368,"nome":"CROSSFOX 1.6 Mi Total Flex 8V 5p"},{"codigo":5983,"nome":"CROSSFOX I MOTION 1.6 Mi T. Flex 8V 5p"},
{"codigo":10368,"nome":"Delivery Express Turbo Diesel (E6)"},{"codigo":10137,"nome":"Delivery Express+ Turbo Diesel"},{"codigo":4857,"nome":"EOS Cab. 2.0 Turbo FSI Tiptronic"},{"codigo":2369,"nome":"Eurovan 2.4 Diesel"},{"codigo":5082,"nome":"Fox 1.0 Mi Total Flex 8V 3p"},{"codigo":5083,"nome":"Fox 1.0 Mi Total Flex 8V 5p"},{"codigo":5084,"nome":"Fox 1.6 Mi I MOTION Total Flex 8V 5p"},{"codigo":5085,"nome":"Fox 1.6 Mi Total Flex 8V 5p"},{"codigo":6548,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 3p"},
{"codigo":6549,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 5p"},{"codigo":6039,"nome":"Fox BlueMotion 1.6 Mi Total Flex 3p."},{"codigo":6040,"nome":"Fox BlueMotion 1.6 Mi Total Flex 5p."},
{"codigo":2370,"nome":"Fox City 1.0 Mi/ 1.0Mi Total Flex 8V 5p"},{"codigo":2371,"nome":"Fox City 1.0Mi/ 1.0Mi Total Flex 8V 3p"},{"codigo":7269,"nome":"Fox Comfortline 1.0 Flex 12V 5p"},{"codigo":6934,"nome":"Fox Comfortline 1.0 Flex 8V 5p"},{"codigo":6935,"nome":"Fox Comfortline 1.6 Flex 8V 5p"},
{"codigo":6936,"nome":"Fox Comfortline I Motion 1.6 Flex 8V 5p"},{"codigo":8112,"nome":"Fox Connect 1.6 Flex 8V 5p"},{"codigo":8133,"nome":"Fox Connect I Motion 1.6 Flex 8V 5p"},{"codigo":4668,"nome":"Fox extreme 1.6 Mi Total Flex 8V 5p"},{"codigo":6937,"nome":"Fox Highline I MOTION 1.6 Flex 16V 5p"},
{"codigo":6938,"nome":"Fox Highline1.6 Flex 16V 5p"},{"codigo":7172,"nome":"Fox PEPPER 1.6 Flex 16V 5p"},
{"codigo":7173,"nome":"Fox PEPPER I MOTION 1.6 Flex 16V 5p"},{"codigo":2372,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 3p"},{"codigo":2373,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 4p"},
{"codigo":2374,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 3p"},{"codigo":2375,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 4p"},{"codigo":5087,"nome":"Fox PRIME/Hghi. IMOTION 1.6 T.Flex 8V 5p"},

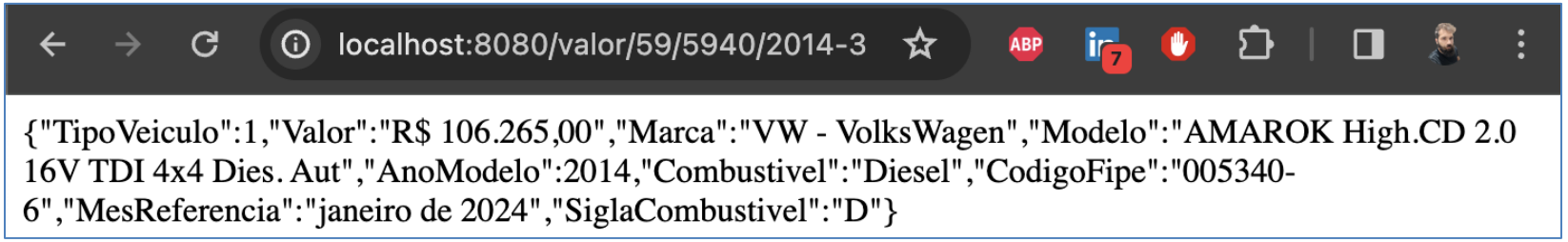


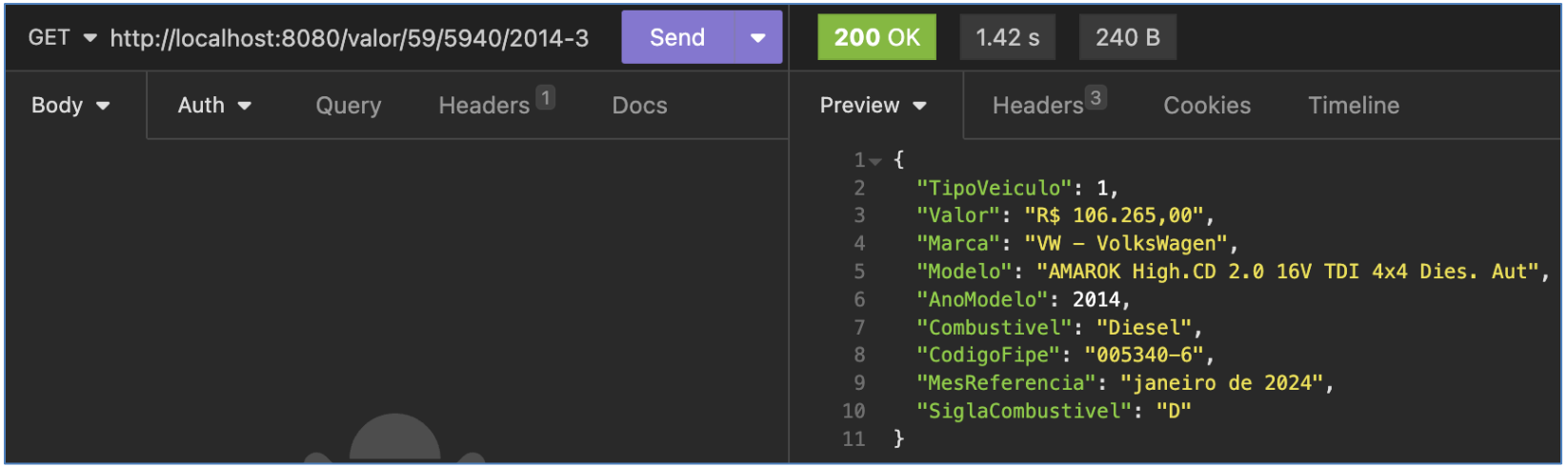
Anos


localhost:8080/anos/59/5940
[{"codigo":"2022-3","nome":"2022 Diesel"},{"codigo":"2021-3","nome":"2021 Diesel"},
{"codigo":"2020-3","nome":"2020 Diesel"},{"codigo":"2019-3","nome":"2019 Diesel"},
{"codigo":"2018-3","nome":"2018 Diesel"},{"codigo":"2017-3","nome":"2017 Diesel"},
{"codigo":"2016-3","nome":"2016 Diesel"},{"codigo":"2015-3","nome":"2015 Diesel"},
{"codigo":"2014-3","nome":"2014 Diesel"},{"codigo":"2013-3","nome":"2013 Diesel"},
{"codigo":"2012-3","nome":"2012 Diesel"},{"codigo":"2011-3","nome":"2011 Diesel"}]


GET http://localhost:8080/anos/59/59
Send
200 OK
549 ms
493 B
Body
Auth
Query
Headers 1
Docs
Preview
Headers 3
Cookies
Timeline
Enter a URL and send to get a response
Select a body type from above to send data in the body of a request
Introduction to Insomnia
[
{
"codigo": "2022-3",
"nome": "2022 Diesel"
},
{
"codigo": "2021-3",
"nome": "2021 Diesel"
},
{
"codigo": "2020-3",
"nome": "2020 Diesel"
},
{
"codigo": "2019-3",
"nome": "2019 Diesel"
},
{
"codigo": "2018-3",
"nome": "2018 Diesel"
},
{
"codigo": "2017-3",
"nome": "2017 Diesel"
},
{
"codigo": "2016-3",
"nome": "2016 Diesel"
},
{
"codigo": "2015-3",
"nome": "2015 Diesel"
},
{
"codigo": "2014-3",
"nome": "2014 Diesel"
},
{
"codigo": "2013-3",

Valor


localhost:8080/valor/59/5940/2014-3
{"TipoVeiculo":1,"Valor":"R$ 106.265,00","Marca":"VW - VolksWagen","Modelo":"AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut","AnoModelo":2014,"Combustivel":"Diesel","CodigoFipe":"005340-6","MesReferencia":"janeiro de 2024","SiglaCombustivel":"D"}


GET http://localhost:8080/valor/59/5940/2014-3
Send
200 OK
1.42 s
240 B
Body
Auth
Query
Headers 1
Docs
Preview
Headers 3
Cookies
Timeline
{
"TipoVeiculo": 1,
"Valor": "R$ 106.265,00",
"Marca": "VW - VolksWagen",
"Modelo": "AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut",
"AnoModelo": 2014,
"Combustivel": "Diesel",
"CodigoFipe": "005340-6",
"MesReferencia": "janeiro de 2024",
"SiglaCombustivel": "D"
}

Bônus

- Padrão **MVC** - Model View Controller

- O padrão MVC (Model-View-Controller) é um padrão arquitetural amplamente utilizado em desenvolvimento de software, especialmente em frameworks para construção de aplicativos web. Ele separa a aplicação em três componentes principais, cada um com responsabilidades específicas.

Bônus

- Padrão **MVC** - Model View Controller

- **Model (Modelo)**: Representa a camada de dados da aplicação. Responsável por gerenciar o estado da aplicação e realizar operações relacionadas aos dados. Geralmente, interage com o banco de dados ou outras fontes de dados.

- **Exemplo**: Usuario.java, Produto.java, etc.

Bônus

- Padrão **MVC** - Model View Controller

- **View (Visualização)**: Responsável pela apresentação dos dados ao usuário. Exibe as informações do modelo de maneira apropriada para o usuário. Geralmente, não realiza operações lógicas ou manipulações de dados, focando apenas na exibição.

- **Exemplo**: Tela de login (.html), de cadastro, etc.

Bônus

- Padrão **MVC** - Model View Controller

- **Controller (Controlador)**: Recebe e processa as entradas do usuário, como cliques de botões ou solicitações HTTP. Atua como um intermediário entre a camada de modelo e a camada de visualização. Responsável por atualizar o modelo com base nas interações do usuário e atualizar a visualização com base nas mudanças no modelo.

- **Exemplo**: LoginController.java, CarrinhoController.java, etc.

Bônus

- Essa separação em três componentes distintos (Model, View e Controller) torna o código mais modular, facilitando a manutenção e a escalabilidade.

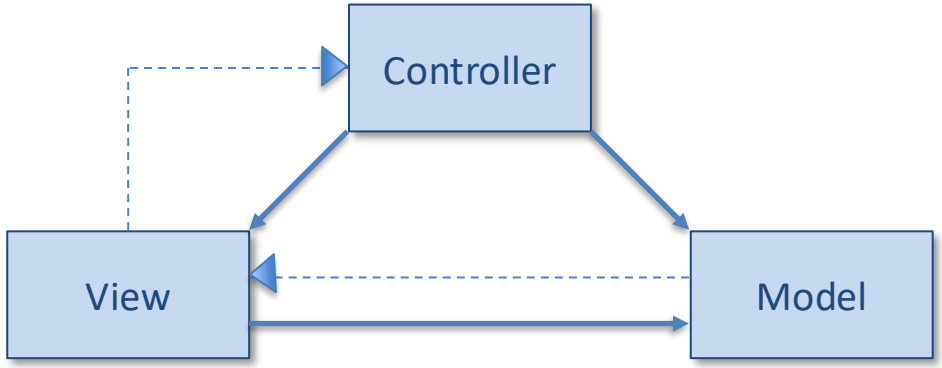
- Além disso, permite que diferentes partes da aplicação evoluam independentemente umas das outras.

Bônus

- Fluxo típico no padrão MVC:
  - 1) O usuário interage com a aplicação por meio da View.
  - 2) A View notifica o Controller sobre a interação do usuário.
  - 3) O Controller processa a interação e atualiza o Model conforme necessário.
  - 4) O Model notifica a View sobre as alterações nos dados.
  - 5) A View atualiza a interface do usuário para refletir as alterações no Model.

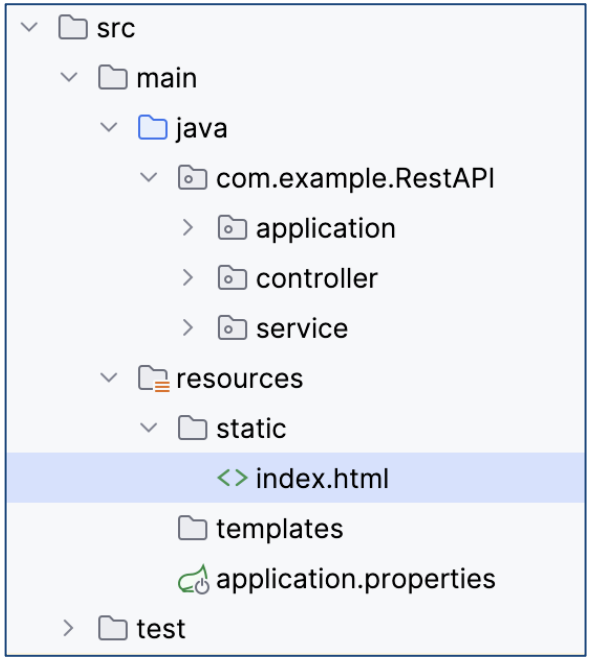
Bônus

- Fluxo típico no padrão MVC:



Bônus

- Esboçando um **front-end**:
  - Vamos inserir agora um **index.html** em uma camada **view** do nosso projeto.
  - No Spring Boot, a pasta **static** é um local especial para armazenar recursos estáticos, como arquivos HTML, CSS, JavaScript, imagens, etc.
  - Lembrando que esta é uma organização do Spring Boot. Em outros tipos de aplicações você poderia criar um mais pacote **view**, no mesmo nível dos pacotes **controller** e **model**.



Bônus

- Quando você coloca arquivos HTML nessa pasta, o Spring Boot automaticamente os trata como recursos estáticos e os disponibiliza diretamente para os clientes (navegadores) sem a necessidade de processamento adicional pelo controlador.

- A razão pela qual o Spring Boot utiliza a pasta static por padrão para recursos estáticos está relacionada à convenção sobre configuração.

Bônus

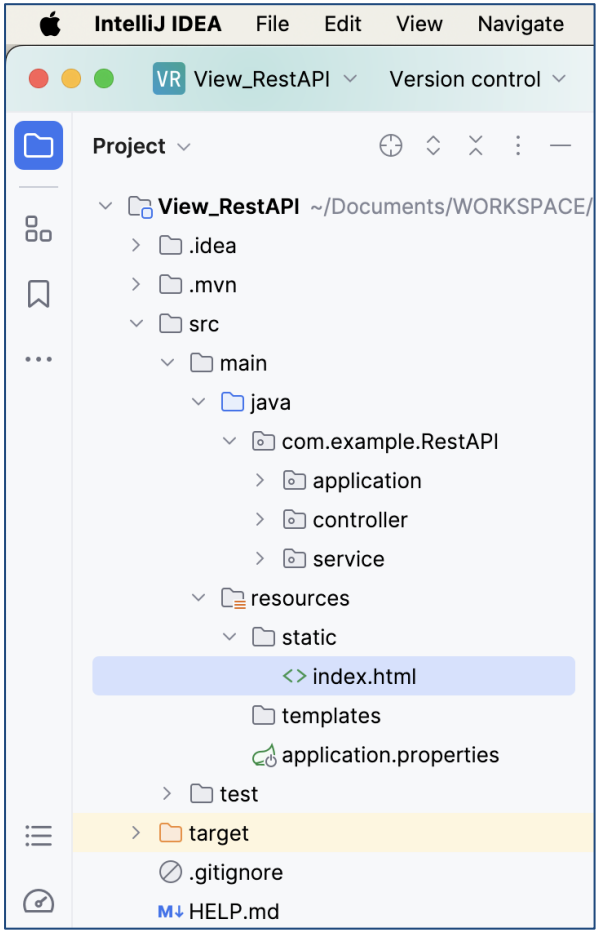
- O Spring Boot segue uma abordagem de "opinião sobre a configuração", o que significa que ele fornece configurações padrão sensíveis e automatizadas para ajudar os devs a começarem rapidamente, sem a necessidade de configurações extensivas.

- O objetivo aqui é que o desenvolvedor foque nas regras de negócio e deixe a configuração chata para o Spring Boot.

Bônus

- Esboçando um **front-end**:
  - Já a pasta **templates** no Spring Boot é um diretório especial designado para armazenar templates de visualização, geralmente usados em conjunto com engines de templates como Thymeleaf, FreeMarker ou Velocity.

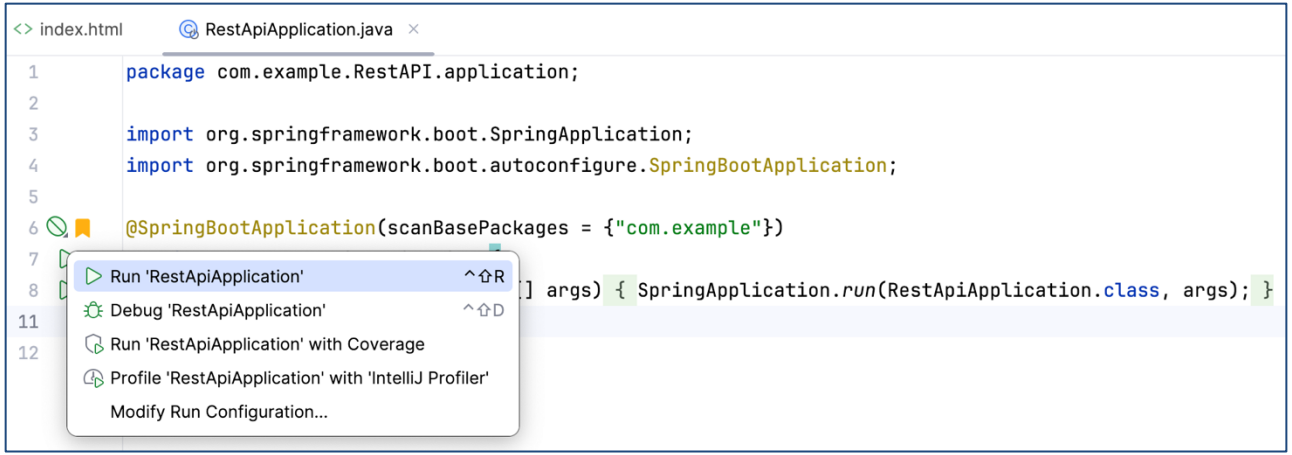
  - O Spring Boot reconhece automaticamente os arquivos presentes nesta pasta e permite a fácil integração entre controladores e a camada de visualização.



Bônus

- Adicione uma tag HTML qualquer ao arquivo **index.html** apenas para teste e, em seguida, execute a aplicação:

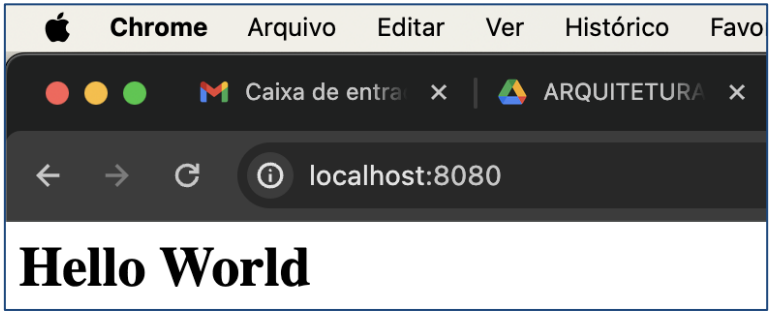




Bônus

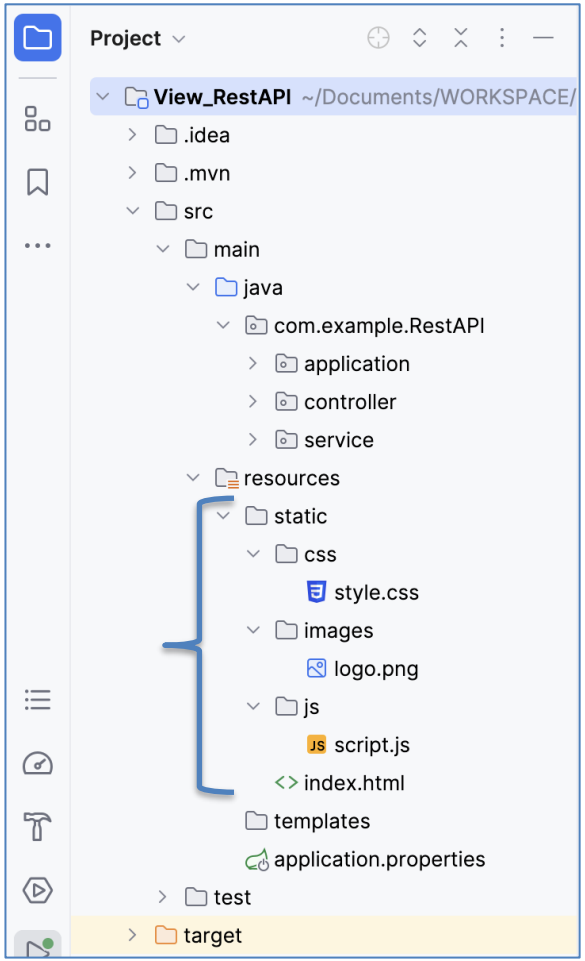
- Por fim, abra seu navegador e acesse a URL:
  - http://localhost:8080/

- Veja que o Spring Boot automaticamente tratou o recurso estático (**index.html**) e o disponibilizou diretamente para o cliente (navegador) sem a necessidade de nenhum código adicional no controlador.



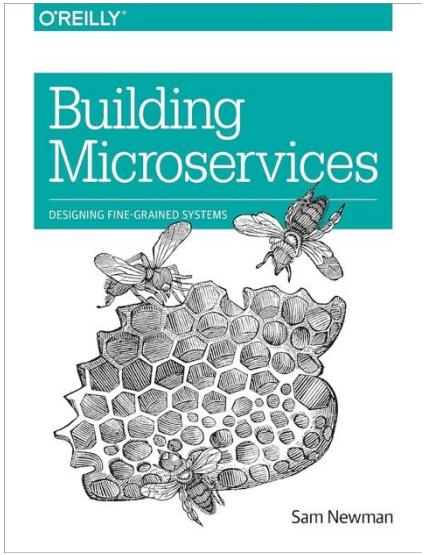
Bônus

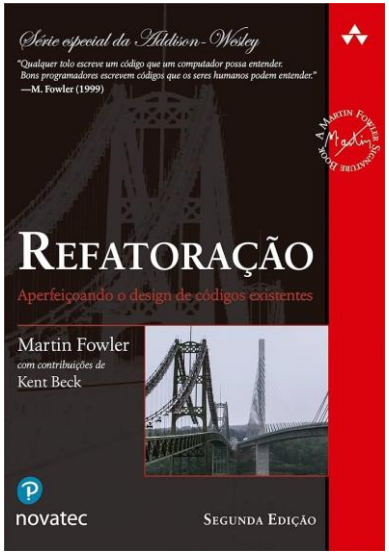
- Esboçando um **front-end**:
  - Caso você queira adicionar uma folha de estilo **style.css**, um arquivo JavaScript **script.js** e uma imagem **logo.png**, esse seria um exemplo de estrutura da pasta **static**.
  - Em seguida, bastaria referenciar esses arquivos na página **index.html**.

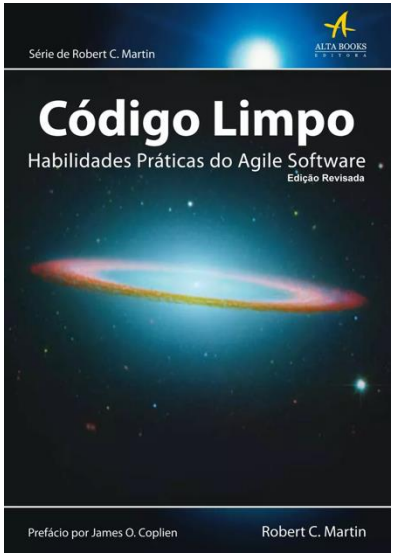


Referências básicas

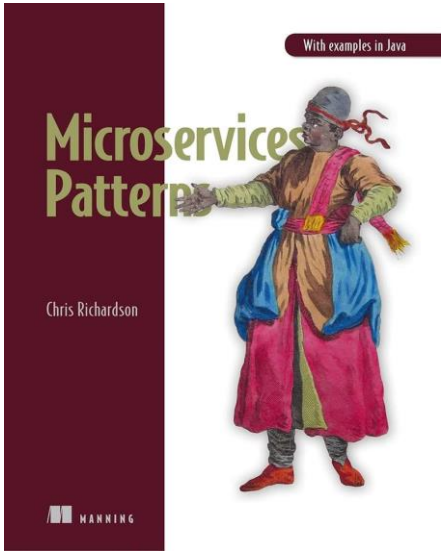


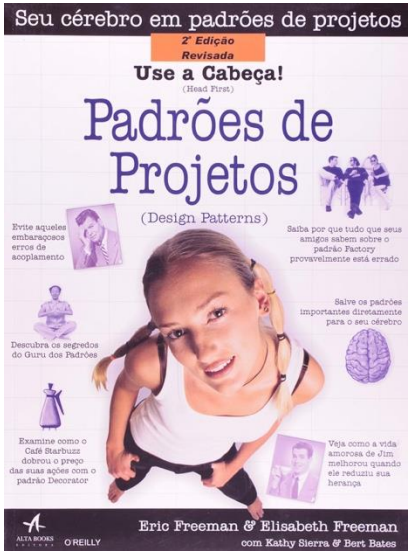


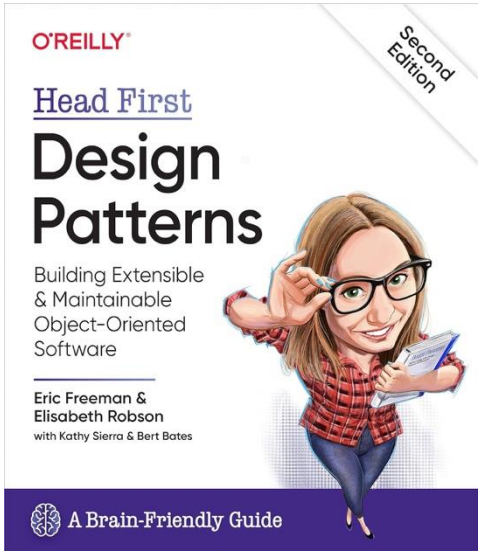


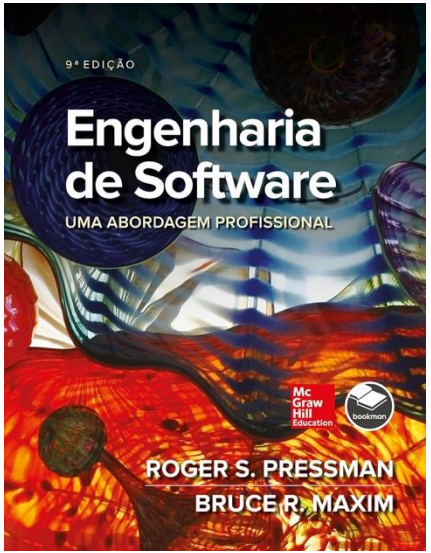


Referências complementares

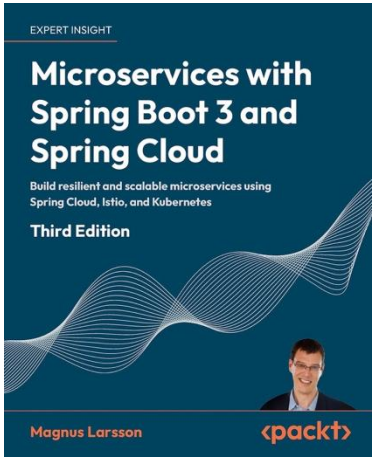




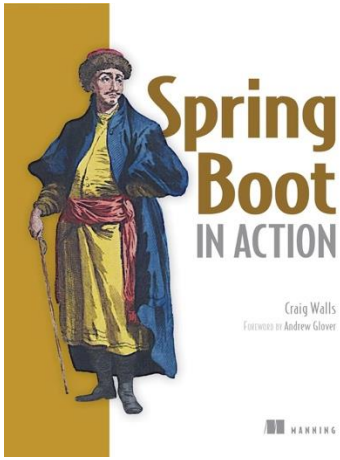


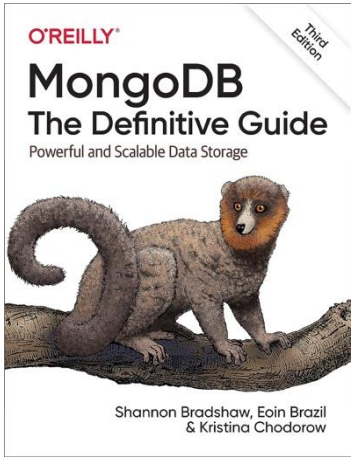


Outras referências

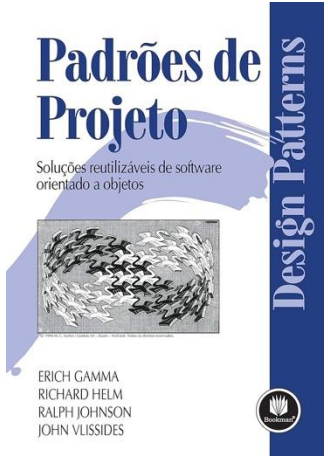












Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes

...